🔐 AWS Security & Compliance — Category 8: Complete Interview Guide

> **Target Audience:** DevOps / SRE / Platform / Cloud Engineers
> **Difficulty:** Beginner → Advanced
> **Topics Covered:** IAM Trust Policies, Permission Boundaries, SCPs, KMS, Secrets Manager, Parameter Store, GuardDuty, Security Hub, Inspector, WAF, Shield, AWS Config, ABAC, Session Policies, Multi-Region Keys, KMS Grants, CloudTrail, VPC Flow Logs, Athena, Macie, Detective, Audit Manager

---

📋 Table of Contents

1. 8.1 IAM Deep Dive — Trust Policies, Permission Boundaries, SCPs
2. 8.2 KMS — CMKs, Key Policies, Envelope Encryption
3. 8.3 Secrets Manager vs Parameter Store
4. 8.4 GuardDuty, Security Hub, Inspector
5. 8.5 WAF & Shield
6. 8.6 AWS Config — Rules, Conformance Packs, Remediation
7. 8.7 IAM — ABAC, Session Policies
8. 8.8 KMS — Multi-Region Keys, Cross-Account Access, Grants
9. 8.9 VPC Flow Logs + CloudTrail + Athena (Security Audit Pattern)
10. 8.10 Macie, Detective, Audit Manager

8.1 IAM Deep Dive — Trust Policies, Permission Boundaries, SCPs

---

🟢 What It Is in Simple Terms

IAM (Identity and Access Management) is AWS's authorization and authentication system. Every API call in AWS goes through IAM — it answers "who is making this call?" and "are they allowed to do this?" Understanding the layers of control is what separates someone who knows IAM from someone who truly understands it.

🔍 The Policy Evaluation Model

```
When an API call arrives, IAM evaluates in this order:
(ALL applicable policies must be checked)

┌──────────────────────────────────────────────────────────────────┐
│  1. Explicit DENY in any policy?  → DENY immediately (full stop)│
```

```
│                                                                  │
│  2. SCPs (Service Control Policies) allow this?                  │
│     → No?  DENY                                                  │
│                                                                  │
│  3. Resource-based policy allows this principal?                 │
│     → Yes? ALLOW (same account — no identity policy needed)      │
│            (cross-account still needs identity policy too)       │
│                                                                  │
│  4. Permission Boundary allows this?                             │
│     → No?  DENY (boundary must permit AND identity must permit)  │
│                                                                  │
│  5. Session Policy (for assumed roles) allows this?              │
│     → No?  DENY                                                  │
│                                                                  │
│  6. Identity-based policy (attached to user/role) allows this?   │
│     → Yes? ALLOW                                                 │
│     → No explicit Allow? → IMPLICIT DENY                         │
└──────────────────────────────────────────────────────────────────┘

Key rule: Default is DENY. An explicit Allow must exist at each relevant
layer.
Key rule: An explicit DENY anywhere = DENY, regardless of any Allow.
```

🧩 Identity Policies

```jsonc
// AWS managed policy: AmazonS3ReadOnlyAccess
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::*", "arn:aws:s3:::*/*"]
  }]
}

// Inline policy with condition keys
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::prod-data/*",
    "Condition": {
      "StringEquals": {
        "s3:prefix":              "reports/",
        "aws:RequestedRegion":    "us-east-1"
      },
      "Bool": {
        "aws:SecureTransport": "true"
      },
```

```
      "DateGreaterThan": {
        "aws:CurrentTime": "2024-01-01T00:00:00Z"
      }
    }
  }]
}
```

```
Condition operators:
├── StringEquals / StringNotEquals
├── StringLike          (supports * and ? wildcards)
├── NumericEquals / NumericLessThan / NumericGreaterThan
├── DateEquals / DateLessThan / DateGreaterThan
├── Bool
├── IpAddress / NotIpAddress
├── ArnEquals / ArnLike
└── Null                (check if condition key exists at all)

Global condition keys (available across all services):
├── aws:CurrentTime          → time-based access windows
├── aws:RequestedRegion      → restrict to specific regions
├── aws:SourceIp             → restrict to IP ranges
├── aws:SecureTransport      → require HTTPS (TLS)
├── aws:MultiFactorAuthPresent → require MFA for the call
├── aws:PrincipalAccount     → the account ID of the caller
├── aws:PrincipalArn         → the full ARN of the calling principal
└── aws:CalledVia            → services calling on your behalf
                               (e.g., cloudformation.amazonaws.com)
```

🧩 Trust Policies — Who Can Assume a Role

```
Trust policy = JSON document attached to an IAM Role that defines who
(which principal) is allowed to assume that role. Who is permitted to use
this role?
Under the hood, assuming a role works via AWS STS (Security Token Service)
— the trust policy controls who is allowed to call sts:AssumeRole on that
role, and in return gets back temporary credentials.

The Two Parts of Any IAM Role
Every IAM Role has two separate policies:
Trust Policy --> Who can assume this role?
Permission Policy --> What can they do after assuming it?

Trust policy structure:
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {          ← WHO can assume this role
```

```
      "Service": "ec2.amazonaws.com"
    },
    "Action": "sts:AssumeRole",
    "Condition": {}         ← optional additional conditions
  }]
}

Principal types in trust policies:
├── AWS account:       "Principal": {"AWS": "arn:aws:iam::123456789:root"}
├── Specific IAM user: "Principal": {"AWS": "arn:aws:iam::123:user/alice"}
├── Specific IAM role: "Principal": {"AWS": "arn:aws:iam::123:role/dev-
role"}
├── AWS service:       "Principal": {"Service": "lambda.amazonaws.com"}
├── Federated:         "Principal": {"Federated": "cognito-
identity.amazonaws.com"}
└── Everyone:          "Principal": "*"  ← very dangerous! always add
conditions

Trust policy with ExternalId (secure cross-account):
{
  "Statement": [{
    "Effect": "Allow",
    "Principal": {"AWS": "arn:aws:iam::PARTNER-ACCT:root"},
    "Action": "sts:AssumeRole",
    "Condition": {
      "StringEquals": {
        "sts:ExternalId": "unique-secret-id-partner-provided"
      },
      "Bool": {
        "aws:MultiFactorAuthPresent": "true"
      }
    }
  }]
}

ExternalId = confused deputy protection:
A third-party vendor should not be able to use their AWS access
to reach YOUR account and also another customer's account using
the same vendor role. ExternalId is a unique secret per relationship.
```

```
# sts:AssumeRole flow
aws sts assume-role \
  --role-arn arn:aws:iam::123:role/cross-account-role \
  --role-session-name alice-session \
  --external-id "unique-customer-external-id"

# Returns: AccessKeyId, SecretAccessKey, SessionToken (temp credentials)
# Temp credentials expire: 1-12 hours (configurable per role)
```

```
Cross-Account Role Assumption — Simple Example

The Scenario

You have two AWS accounts:

  Account A — Production    (ID: 111111111111)
  Account B — DevOps        (ID: 222222222222)

Your DevOps team sits in Account B and needs to
read CloudWatch logs from Account A (Production).

──────────────────────────────────────────────────

Step 1 — Create a Role in Account A (Production)

Trust Policy (who can assume this role):

  {
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::222222222222:root"
    },
    "Action": "sts:AssumeRole"
  }

  --> "Anyone from Account B is allowed to assume this role."

Permission Policy (what they can do after assuming):

  {
    "Effect": "Allow",
    "Action": [
      "logs:DescribeLogGroups",
      "logs:GetLogEvents"
    ],
    "Resource": "*"
  }

──────────────────────────────────────────────────

Step 2 — Grant Permission in Account B

Attach this to your DevOps IAM user/role in Account B:

  {
    "Effect": "Allow",
    "Action": "sts:AssumeRole",
    "Resource": "arn:aws:iam::111111111111:role/ProdReadOnlyRole"
  }

──────────────────────────────────────────────────
```

```
The Handshake — Both Sides Must Agree

  Account B (DevOps)                 Account A (Prod)
  +-----------------+                +------------------+
  |  DevOps User    |                |  ProdReadOnlyRole|
  |                 |   AssumeRole   |                  |
  |                 |--------------->|  Trust Policy    |
  |                 |                |  check ✅        |
  |                 |<---------------|                  |
  |                 |  Temp Creds    |                  |
  +-----------------+  (15min-12hr)  +------------------+

  Both sides must agree:
  [✅] Account A's role must TRUST Account B
  [✅] Account B's user must have PERMISSION to assume that role

  If either side is missing --> Access Denied

──────────────────────────────────────────────────

Step 3 — Assume the Role via CLI

  $ aws sts assume-role \
      --role-arn "arn:aws:iam::111111111111:role/ProdReadOnlyRole" \
      --role-session-name "DevOpsSession"

  Response:
  {
    "AccessKeyId":     "ASIA...",
    "SecretAccessKey": "...",
    "SessionToken":    "...",
    "Expiration":      "2026-03-18T10:00:00Z"
  }

  No permanent keys. No IAM users in prod. Creds auto-expire.

──────────────────────────────────────────────────

Best Practice — Narrow the Trust

  Instead of trusting ALL of Account B:
    "AWS": "arn:aws:iam::222222222222:root"      ⚠  broad

  Trust only a SPECIFIC role in Account B:
    "AWS": "arn:aws:iam::222222222222:role/DevOpsRole"  ✅  tight

  This limits access to just that one role,
  not every user/service in the account.
```

```
Role chaining:
├── Role A assumes Role B (a role assuming another role)
```

```
├── Maximum session duration: 1 hour for chained roles
│   (regardless of individual role max session settings)
├── Credentials from chained roles cannot further chain
└── Avoid deep chains — increases blast radius and complexity
```

🧩 Permission Boundaries

```
Permission Boundary = maximum permissions a principal can EVER have
                      Even if identity policy grants more, boundary caps it

Identity policy grants: Allow *  (all AWS actions)
Permission boundary allows: s3:* and dynamodb:* only
Effective permissions:      S3 and DynamoDB only

The effective permission = intersection of:
Identity Policy ∩ Permission Boundary (∩ Session Policy if set)

┌──────────────────────────────────────────────────────────────┐
│  Identity Policy:     S3 + DynamoDB + EC2 + Lambda           │
│                                                              │
│  Permission Boundary: S3 + DynamoDB only                     │
│                                                              │
│  Effective:           S3 + DynamoDB  ← intersection only     │
└──────────────────────────────────────────────────────────────┘

Primary use case: safe delegation of IAM permission creation

Problem:
  Developer team needs to create IAM roles for their Lambda functions.
  Granting iam:CreateRole + iam:AttachRolePolicy is dangerous —
  developers could create an admin role and escalate their own privileges.

Solution: Require permission boundaries on all developer-created roles.

Policy allowing developers to create roles (with boundary enforcement):
{
  "Statement": [{
    "Effect": "Allow",
    "Action": ["iam:CreateRole", "iam:AttachRolePolicy",
"iam:PutRolePolicy"],
    "Resource": "arn:aws:iam::*:role/dev-*",
    "Condition": {
      "StringEquals": {
        "iam:PermissionsBoundary":
          "arn:aws:iam::123:policy/DeveloperBoundary"
      }
    }
  }]
}
```

```
Developers MUST attach DeveloperBoundary when creating any role.
Even if they create a role named "admin", the boundary limits it.
The dev team can self-service Lambda roles safely without privilege
escalation.

The condition key iam:PermissionsBoundary is the enforcement mechanism —
without this condition on the developer's iam:CreateRole permission,
developers can create roles without any boundary attached, defeating the
entire purpose.
```


```
The condition key iam:PermissionsBoundary is the enforcement mechanism —
without this condition on the developer's iam:CreateRole permission,
developers can create roles without any boundary attached, defeating the
entire purpose.
```


```
aws iam create-role \
  --role-name dev-lambda-processor \
  --assume-role-policy-document file://trust.json \
  --permissions-boundary arn:aws:iam::123:policy/DeveloperBoundary
```

🧩 SCPs — Service Control Policies

```
SCPs = AWS Organizations-level policies
       Applied to: Organization root, OUs, individual accounts
       Purpose: restrict what ANY principal in the account can EVER do

SCPs do NOT grant permissions — they ONLY restrict them.
Even an account's root user is restricted by SCPs from parent OUs!

SCP evaluation:
Account admin tries to: delete S3 bucket
SCP says: Deny s3:DeleteBucket for this OU
Result: DENIED — even though the admin has full S3 permissions locally

SCP hierarchy:
Organization Root SCP
└── OU: Production
    └── SCP: Deny non-approved regions
        └── Account: prod-us-east-1
            └── SCP: Deny disabling GuardDuty
```

```
// 1. Deny all actions in non-approved regions
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
```

```
    "Condition": {
      "StringNotEquals": {
        "aws:RequestedRegion": ["us-east-1", "us-east-2", "eu-west-1"]
      }
    }
  }

  // 2. Protect security baseline — deny disabling security services
  {
    "Effect": "Deny",
    "Action": [
      "guardduty:DeleteDetector",
      "guardduty:DisassociateFromMasterAccount",
      "cloudtrail:DeleteTrail",
      "cloudtrail:StopLogging",
      "config:DeleteConfigurationRecorder",
      "securityhub:DisableSecurityHub"
    ],
    "Resource": "*"
  }

  // 3. Require MFA for deleting MFA devices
  {
    "Effect": "Deny",
    "Action": ["iam:DeleteVirtualMFADevice", "iam:DeactivateMFADevice"],
    "Resource": "*",
    "Condition": {
      "BoolIfExists": {"aws:MultiFactorAuthPresent": "false"}
    }
  }

  // 4. Deny expensive services in dev/test accounts
  {
    "Effect": "Deny",
    "Action": ["redshift:CreateCluster", "sagemaker:CreateTrainingJob"],
    "Resource": "*"
  }
```

```
⚠ SCPs do NOT apply to:
├── The management (root) account of the organization
├── Service-linked roles
└── Actions performed by AWS services themselves (e.g., CloudFormation)

SCPs are guardrails — not grants.
Even FullAdmin IAM policy + SCP Deny = DENIED.
```

🧩 IAM Access Analyzer

```
IAM Access Analyzer = automated tool that identifies resources in your
account
                      (or organization) that are shared with EXTERNAL
principals

"External" means: outside your AWS account or outside your AWS
Organization

Resources Access Analyzer monitors:
├── S3 buckets
├── IAM roles (trust policies allowing external assume)
├── KMS keys (key policies granting cross-account access)
├── Lambda functions and layers (resource-based policies)
├── SQS queues
├── Secrets Manager secrets
└── SNS topics

How it works:
├── You define a ZONE OF TRUST: your account OR your entire organization
├── Access Analyzer continuously monitors resource-based policies
├── Any resource accessible from OUTSIDE the zone of trust → FINDING
└── Findings appear in Security Hub + EventBridge for automated response

Finding types:
├── External access findings:
│   "S3 bucket prod-data is publicly accessible via bucket policy"
│   "IAM role cross-account-role trusts account 999888777 (not in org)"
│   "KMS key allows Decrypt from arn:aws:iam::EXTERNAL-ACCT:root"
└── Unused access findings (IAM Access Analyzer for unused access):
    "IAM role has not used EC2 permissions in 90 days"
    "IAM user has S3:DeleteObject permission never used in 180 days"
    → Use to right-size permissions (reduce blast radius)

Zone of trust options:
├── Account zone:      external = anything outside this AWS account
└── Organization zone: external = anything outside this AWS Organization
                       (cross-account within org = NOT flagged as finding)

Archive rules — suppress expected findings:
{
  "filter": [
    {"criterion": "principal.AWS", "eq": ["arn:aws:iam::PARTNER-
ACCT:root"]},
    {"criterion": "resource", "contains": ["arn:aws:s3:::partner-shared-
bucket"]}
  ]
}
→ Expected external access: archive the finding (won't re-alert)
→ Unexpected external access: active finding requiring investigation
```

```
# Create an analyzer scoped to the entire organization
aws accessanalyzer create-analyzer \
  --analyzer-name org-external-access \
  --type ORGANIZATION

# List all active findings
aws accessanalyzer list-findings \
  --analyzer-arn arn:aws:accessanalyzer:us-east-1:123:analyzer/org-
external-access \
  --filter '{"status": {"eq": ["ACTIVE"]}}'

# Archive an expected finding (known/intentional external access)
aws accessanalyzer update-findings \
  --analyzer-arn arn:aws:accessanalyzer:... \
  --ids ["finding-id-123"] \
  --status ARCHIVED
```

```
Access Analyzer policy validation:
Before deploying a policy, validate it against IAM best practices:

aws accessanalyzer validate-policy \
  --policy-type IDENTITY_POLICY \
  --policy-document file://my-policy.json

Returns findings like:
├── SECURITY_WARNING: "S3 bucket wildcard — consider scoping to specific
bucket"
├── WARNING:          "Unused condition key in this context"
├── SUGGESTION:       "Consider adding condition to restrict to specific
region"
└── ERROR:            "Invalid syntax in policy statement"

Access Analyzer policy generation:
├── Monitors CloudTrail for 90 days
├── Generates a LEAST-PRIVILEGE policy based on ACTUAL observed API calls
└── Use to right-size overly broad existing policies

aws accessanalyzer start-policy-generation \
  --policy-generation-details '{"principalArn": "arn:aws:iam::123:role/my-
role"}' \
  --cloud-trail-details '{
    "accessRole": "arn:aws:iam::123:role/analyzer-role",
    "trailArn": "arn:aws:cloudtrail:us-east-1:123:trail/my-trail",
    "startTime": "2024-01-01T00:00:00Z"
  }'
```

IAM Access Advisor

```
Access Advisor = per-principal view of LAST ACCESSED timestamps
                 for every service and action a principal has permission
to use

Shows: "This role has permission to call 47 AWS services.
        It has only actually called 3 of them in the last 90 days."

Access at two levels of granularity:
├── Service level:  last time the principal accessed ANY action in the
service
└── Action level:   last time a SPECIFIC action was called
    (available for S3, IAM, Lambda, SQS — not all services yet)

Use cases:
├── Right-sizing permissions:
│   Role has EC2, S3, DynamoDB, Lambda permissions.
│   Access Advisor: EC2 last used 400 days ago.
│   → Safe to remove EC2 permissions (likely never needed them)
│
├── Identifying unused IAM users/roles:
│   No service access in 180+ days → candidate for deletion
│
└── Enforcing least privilege:
    Before doing a quarterly access review, pull Access Advisor data
    to show which permissions are actively used vs dormant

Access the data:
# In IAM console: click any user/role → "Access Advisor" tab
# API:
aws iam generate-service-last-accessed-details \
  --arn arn:aws:iam::123:role/my-role

aws iam get-service-last-accessed-details \
  --job-id <job-id-from-above>
```

```
Access Analyzer vs Access Advisor:
├── Access Analyzer: WHO from OUTSIDE can reach your resources?
│   → Finds unintended external access → close the gap
└── Access Advisor:  WHAT permissions is this principal actually using?
    → Finds unused permissions → remove the excess

Both are least-privilege tools — different direction of analysis.
Access Analyzer = inbound exposure analysis.
Access Advisor  = outbound usage analysis.
```

💬 Short Crisp Interview Answer

"IAM policy evaluation follows a strict hierarchy: explicit DENY wins over everything first, then SCPs restrict what accounts in an org can ever do (even root), then resource-based policies, then permission boundaries (which cap what an identity policy can grant — the effective permission is the intersection of the two), then session policies, then identity-based policies. Trust policies define who can assume a role — they're resource-based policies on the role itself with sts:AssumeRole as the action. Permission boundaries are the key tool for delegating IAM management safely: developers can create roles for their Lambda functions but can't escalate beyond what the boundary allows. SCPs are organizational guardrails — they can deny region usage, protect security services from being disabled, and restrict expensive services in dev accounts. IAM Access Analyzer continuously monitors resource-based policies to find any resource accessible from outside your zone of trust — flagging unexpected cross-account S3 buckets, role trust policies, and KMS key access. Access Advisor complements it by showing which permissions a principal has actually used in the last 90 days, making it the primary tool for right-sizing overly broad policies toward least privilege."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Explicit Deny always wins	Cannot override an explicit Deny with any Allow anywhere
SCPs restrict root too	Even account root user is blocked by SCPs from parent OU
Permission boundary ≠ grant	Having a boundary doesn't grant anything — identity policy must also allow
Role chaining 1-hour limit	Chained role sessions max out at 1 hour regardless of individual role settings
ExternalId confusion	Customer sets ExternalId and shares it with the vendor — not set by the vendor
Trust policy is mandatory	No trust policy = role exists but is completely unusable

- **Region restriction SCPs must use NotAction not Action: "*"** — global services (IAM, Route53, CloudFront, STS) don't populate aws:RequestedRegion. Using Action: * with a region condition blocks these services entirely even in approved regions.
- **Break-glass role exception is not optional for production** — a bad SCP with no break-glass can permanently lock out an entire OU. Always include a specific admin role ARN exclusion condition before attaching any deny SCP to production OUs.

8.2 KMS — CMKs, Key Policies, Envelope Encryption

🟢 What It Is in Simple Terms

KMS (Key Management Service) is AWS's managed encryption key service. You never see the raw cryptographic key material — KMS manages it in FIPS 140-2 validated Hardware Security Modules (HSMs). Every AWS service that encrypts data uses KMS under the hood.

🧩 Key Types

```
AWS Managed Keys (aws/service-name):
├── Created automatically when you enable encryption on a service
│   Examples: aws/s3, aws/ebs, aws/lambda, aws/rds
├── Fully managed by AWS — you cannot modify the key policy
├── No cost (included in service pricing)
├── Rotated automatically every year by AWS
└── Cannot be used cross-account or for custom cross-service encryption

Customer Managed Keys (CMKs):
├── You create and manage them
├── You control the key policy (who can use and who can manage)
├── Cost: $1/key/month + $0.03/10,000 API calls
├── Optional: enable annual automatic key rotation
└── Use for: custom access control, cross-account sharing, audit
requirements

AWS Owned Keys:
├── AWS creates and manages completely (no visibility to you)
├── Shared across multiple AWS customers for internal AWS use
└── Used by services like DynamoDB when no CMK is specified

Key material origin options:
├── AWS generated:    KMS creates and stores key material in HSM
├── External import:  you import your own key material (BYOK)
│   ⚠ You are responsible for the security of the source key material
└── AWS CloudHSM:     key material stored in your dedicated CloudHSM
cluster
    (FIPS 140-2 Level 3 vs Level 2 for standard KMS HSMs)
```

🧩 Key Policies

```
Key Policy = resource-based policy attached directly to the KMS key
             Required for CMKs — unlike IAM, key policy MUST exist

⚠ KMS requires key policy to explicitly allow access.
   Even an account admin CANNOT use a CMK without key policy permission.
   A CMK with no key policy = completely inaccessible to everyone!

Default key policy (created via console):
{
  "Statement": [
    {
      "Sid": "Enable IAM User Permissions",
      "Effect": "Allow",
      "Principal": {"AWS": "arn:aws:iam::123456789:root"},
      "Action": "kms:*",
```

```
      "Resource": "*"
    }
  ]
}

This root entry allows IAM policies on users/roles to further control
access.

Key policy roles:
├── Key administrators: can manage the key (update policy, schedule
deletion)
│   Actions: kms:Create*, kms:Describe*, kms:Enable*, kms:List*,
│            kms:Put*, kms:Update*, kms:Revoke*, kms:Disable*,
│            kms:ScheduleKeyDeletion, kms:CancelKeyDeletion
│
└── Key users: can use the key for cryptographic operations
    Actions: kms:Encrypt, kms:Decrypt, kms:ReEncrypt*,
             kms:GenerateDataKey*, kms:DescribeKey
```

```
// Cross-account key access — Step 1: key policy in key-owner account
{
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::CONSUMER-ACCT:root"},
  "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
  "Resource": "*"
}
```

```
# Cross-account key access — Step 2: IAM policy in consumer account
# (both steps are required — neither alone is sufficient)
aws iam put-role-policy \
  --role-name consumer-role \
  --policy-name use-shared-key \
  --policy-document '{
    "Statement": [{
      "Effect": "Allow",
      "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
      "Resource": "arn:aws:kms:us-east-1:KEY-ACCT:key/key-id"
    }]
  }'
```

🧩 Envelope Encryption

```
Problem: KMS can only encrypt data up to 4KB directly.
         Real application data (files, database records) is much larger.

Solution: Envelope Encryption — encrypt a key with another key
```

```
Step-by-step flow:
1. Your app calls KMS: GenerateDataKey(KeyId=your-cmk)
2. KMS returns two things:
   ├── Plaintext DEK  (Data Encryption Key) — use this to encrypt data
   └── Encrypted DEK  — the plaintext DEK encrypted under your CMK
3. Your app:
   ├── Encrypts data using Plaintext DEK (AES-256, done locally)
   ├── Stores Encrypted DEK alongside the encrypted ciphertext
   └── Discards Plaintext DEK from memory immediately
4. To decrypt later:
   ├── Retrieve the Encrypted DEK from storage
   ├── Call KMS: Decrypt(CiphertextBlob=EncryptedDEK)
   ├── KMS returns Plaintext DEK
   └── Use Plaintext DEK to decrypt data locally


┌──────────────────────────────────────────────────────────────┐
│                    Encrypt Flow                              │
│                                                              │
│  Your App ──GenerateDataKey──► KMS (CMK)                     │
│              ◄── PlaintextDEK + EncryptedDEK                 │
│                                                              │
│  Your App: Encrypt(data, PlaintextDEK) → ciphertext          │
│  Store:    ciphertext + EncryptedDEK   → together            │
│  Discard:  PlaintextDEK               → immediately          │
│                                                              │
│                    Decrypt Flow                              │
│                                                              │
│  Retrieve: ciphertext + EncryptedDEK                         │
│  Your App ──Decrypt(EncryptedDEK)──► KMS (CMK)               │
│              ◄── PlaintextDEK                                │
│  Your App: Decrypt(ciphertext, PlaintextDEK) → plaintext     │
└──────────────────────────────────────────────────────────────┘

Benefits:
├── Data never leaves your application unencrypted
├── Each object can have a unique DEK (breach = only one object exposed)
├── KMS API calls are minimized (one call per object, not per byte)
└── CMK never leaves KMS HSM — only DEKs are transmitted over network

GenerateDataKey vs GenerateDataKeyWithoutPlaintext:
├── GenerateDataKey:                returns plaintext + encrypted DEK
│   Use for: immediate encryption operations
└── GenerateDataKeyWithoutPlaintext: returns only encrypted DEK
    Use for: pre-generating keys for deferred use (e.g., at build time)
```

🧩 Key Rotation

```
Automatic rotation:
├── AWS Managed Keys: always on (rotated every year automatically)
```

```
  ├── Customer Managed Keys: optional (enable it per key)
  ├── Rotation period: 365 days — cannot change this interval
  ├── Key ID, ARN, and aliases stay the SAME after rotation
  ├── Old key material kept indefinitely to decrypt pre-rotation ciphertext
  └── New key material used for all new encryption operations

  aws kms enable-key-rotation --key-id arn:aws:kms:us-east-1:123:key/...

  ⚠ Rotation applies to key MATERIAL only — not the key ID or ARN.
     All references (aliases, policies, application configs) remain valid.
     No re-encryption needed — old ciphertext still decryptable with old
  material.

  Key deletion:
  ├── Minimum 7-day waiting period before deletion executes
  ├── Maximum 30-day waiting period
  ├── Cannot delete immediately — safety buffer to prevent accidents
  ├── Cancel deletion anytime during the waiting period
  └── After deletion completes: ALL ciphertext encrypted with that key
      is PERMANENTLY UNRECOVERABLE — there is no undo
      ⚠ This is a one-way, irreversible action.
  schedule-key-deletion disables the key immediately upon execution — not
  after the waiting period. The waiting period is time to cancel, not a
  grace period before impact. Applications using the key start failing
  within minutes of the command being run, not after 7 days.

  Minimum 7-day window exists specifically for cancellation — use cancel-
  key-deletion + enable-key to recover. After permanent deletion, no
  recovery is possible under any circumstance.
```

💬 Short Crisp Interview Answer

"KMS manages cryptographic keys in FIPS-validated HSMs — you never see raw key material. Customer Managed Keys give you full control over key policy, rotation, and cross-account access. Key policies are mandatory resource-based policies on the key — unlike IAM, no key policy means no access, even for account admins. Envelope encryption solves the 4KB limit: call GenerateDataKey to get a plaintext and encrypted DEK, encrypt your data locally with the plaintext DEK, store only the encrypted DEK alongside your ciphertext, then discard the plaintext DEK immediately. To decrypt, call KMS:Decrypt on the encrypted DEK to recover it, then decrypt locally. Each object gets a unique DEK, so compromising one object's DEK doesn't expose others."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
No key policy = no access	CMK without explicit key policy is completely inaccessible, including to admins

Gotcha	Detail
Key deletion is irreversible	Deleted CMK = all ciphertext encrypted with it is permanently lost forever
Minimum 7-day deletion wait	Cannot delete immediately — cannot be shortened below 7 days
Cross-account = 2 steps required	Key policy alone is not enough — IAM policy in consumer account also required
Rotation keeps old material	Old key material is never deleted after rotation — needed to decrypt old ciphertext
BYOK deletion responsibility	You are responsible for securing the source key material you imported

---

8.3 Secrets Manager vs Parameter Store

🟢 What It Is in Simple Terms

Both services store secrets (database passwords, API keys, certificates) so applications don't hardcode credentials. Secrets Manager is the full-featured service with automatic rotation. Parameter Store is simpler and cheaper, good for configuration values and secrets that don't need rotation.

⚙️ Deep Comparison

```
┌──────────────────────┬──────────────────────┬──────────────────────┐
│ Feature              │ Secrets Manager      │ Parameter Store      │
├──────────────────────┼──────────────────────┼──────────────────────┤
│ Purpose              │ Secrets + rotation   │ Config + secrets     │
│ Automatic rotation   │ ✅ Native, Lambda    │ ❌ Custom only       │
│ Encryption           │ Always KMS           │ SecureString = KMS   │
│                      │                      │ String = plaintext   │
│ Cost                 │ $0.40/secret/month   │ Standard: Free       │
│                      │ + $0.05/10K API calls│ Advanced: $0.05/param│
│ Secret size          │ Up to 64KB           │ Up to 4KB (standard) │
│                      │                      │ Up to 8KB (advanced) │
│ Versioning           │ ✅ AWSCURRENT/AWSPREV│ ✅ Yes               │
│ Cross-account access │ ✅ Resource policy   │ Limited              │
│ Parameter hierarchy  │ ❌                   │ ✅ /app/prod/db/pass │
│ Multi-region replication│ ✅ Yes             │ ❌ No                │
└──────────────────────┴──────────────────────┴──────────────────────┘

Use Secrets Manager for:
├── Database passwords (native RDS/Aurora/Redshift rotation support)
├── API keys that must be rotated automatically on a schedule
├── Credentials shared across multiple services (cross-account)
└── Anything requiring automatic rotation for compliance
```

```
Use Parameter Store for:
├── Application configuration values (not just secrets)
├── Feature flags and environment variables
├── Non-sensitive config (use String tier — completely free)
├── Secrets without rotation needs (use SecureString + KMS)
└── Hierarchical config: /app/prod/db-host, /app/prod/db-port
```

🧩 Secrets Manager — Rotation Deep Dive

```
Rotation flow (RDS password example):

┌──────────────────────────────────────────────────────────────┐
│  Secrets Manager triggers rotation Lambda on schedule        │
│                                                              │
│  Lambda (rotation function) — four mandatory steps:          │
│  Step 1: createSecret  → generate new random password        │
│  Step 2: setSecret     → set new password on RDS database    │
│  Step 3: testSecret    → verify new password connects to RDS │
│  Step 4: finishSecret  → promote new version to AWSCURRENT   │
│          (old AWSCURRENT becomes AWSPREVIOUS)                │
│                                                              │
│  During rotation:                                            │
│  AWSPENDING  = new password (being tested)                   │
│  AWSCURRENT  = current active password (still in use)        │
│                                                              │
│  After rotation completes:                                   │
│  AWSCURRENT  = new password (all new connections use this)   │
│  AWSPREVIOUS = old password (in-flight connections still OK) │
└──────────────────────────────────────────────────────────────┘
```

```bash
aws secretsmanager create-secret \
  --name "prod/myapp/db-password" \
  --secret-string '{"username":"admin","password":"SuperSecretPwd!"}' \
  --kms-key-id arn:aws:kms:...:key/...

# Enable automatic rotation every 30 days
aws secretsmanager rotate-secret \
  --secret-id "prod/myapp/db-password" \
  --rotation-lambda-arn arn:aws:lambda:...:function:SecretsManagerRotation
\
  --rotation-rules AutomaticallyAfterDays=30
```

```python
# Reading secret in application code
import boto3, json

client = boto3.client('secretsmanager')
```

```
response = client.get_secret_value(
    SecretId='prod/myapp/db-password',
    VersionStage='AWSCURRENT'   # always get the active version
)
secret = json.loads(response['SecretString'])
db_password = secret['password']

# ⚠ Cache secrets in application — do NOT call Secrets Manager per
request!
# Cache TTL: ~5 minutes (AWS SDK provides a built-in caching client)
# Rotation handling: detect 401 from DB → refresh secret → retry
connection
```

🧩 Parameter Store — Hierarchy and Tiers

```
# Create hierarchical parameters
aws ssm put-parameter \
  --name "/prod/myapp/db/host" \
  --value "prod-db.cluster.rds.amazonaws.com" \
  --type String                    # plaintext — not sensitive

aws ssm put-parameter \
  --name "/prod/myapp/db/password" \
  --value "SuperSecretPwd!" \
  --type SecureString \
  --key-id arn:aws:kms:...:key/... \
  --tier Advanced

# Get all parameters for prod environment in one API call
aws ssm get-parameters-by-path \
  --path /prod/myapp/ \
  --recursive \
  --with-decryption
```

```
Parameter Store hierarchy example:
/myapp/
├── /myapp/prod/
│   ├── /myapp/prod/db/host     → "prod-db.cluster.rds.amazonaws.com"
│   ├── /myapp/prod/db/port     → "5432"
│   ├── /myapp/prod/db/password → SecureString (KMS encrypted)
│   └── /myapp/prod/feature-x  → "true"
└── /myapp/staging/
    ├── /myapp/staging/db/host  → "staging-db.cluster.rds.amazonaws.com"
    └── /myapp/staging/db/password → SecureString

Parameter tiers:
├── Standard: free, up to 10,000 parameters, 4KB max value size
├── Advanced: $0.05/parameter/month, up to 100,000 parameters, 8KB max
```

```
  │            + Parameter policies (TTL expiration, event notifications)
  └── Intelligent-Tiering: auto-promotes standard to advanced when needed

Parameter policies (advanced tier only):
  ├── Expiration:             auto-delete parameter after a specific date
  ├── ExpirationNotification: SNS alert N days before parameter expires
  └── NoChangeNotification:   alert if parameter unchanged for N days
      (useful for detecting stale passwords that should be rotated)
```

💬 Short Crisp Interview Answer

"Both Secrets Manager and Parameter Store store sensitive values, but they're designed for different use cases. Secrets Manager is built for secrets with automatic rotation — it has native integration with RDS, Aurora, and Redshift to rotate database passwords through a four-step Lambda process (create, set, test, finish) without any downtime. It costs $0.40/secret/month. Parameter Store is more of a general configuration store — free for standard parameters, supports hierarchical paths like /app/prod/db/host, great for mixing config values and secrets. For database credentials and anything requiring automatic rotation or cross-account resource-policy access, use Secrets Manager. For application config that doesn't need rotation, use Parameter Store."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Never call Secrets Manager per request	Cache secrets in app — calling per request is expensive and adds latency
Rotation window = dual-version period	App must handle both AWSPREVIOUS (in-flight connections) and AWSCURRENT
Parameter Store String = plaintext	Always use SecureString for sensitive values — String stores in plaintext
Parameter size limits	Standard: 4KB, Advanced: 8KB — large certificates need Secrets Manager
Cross-account Parameter Store	Limited support — Secrets Manager has proper resource-based policy for cross-account

- **The rotation Lambda has four mandatory steps** — createSecret → setSecret → testSecret → finishSecret. If any step fails, the secret is left in AWSPENDING state and rotation stalls. Monitor rotation failures via CloudWatch Events on the rotation Lambda.
- **Parameter Store is not inferior — it's differently scoped** — use it for configuration values, feature flags, and secrets that don't require rotation or cross-account sharing. Using Secrets Manager for everything is over-engineering and unnecessary cost.

8.4 GuardDuty, Security Hub, Inspector

🟢 What It Is in Simple Terms

These three services form AWS's core security detection stack. GuardDuty is your always-on threat detector — finds active attacks in progress. Security Hub is your centralized security posture dashboard — aggregates findings from all security services. Inspector is your vulnerability scanner — finds known CVEs before they get exploited.

🧩 Amazon GuardDuty

```
GuardDuty = ML-powered continuous threat detection
            Data sources analyzed automatically:
            ├── VPC Flow Logs
            ├── CloudTrail management events
            ├── CloudTrail S3 data events
            ├── DNS query logs
            ├── EKS audit logs and runtime activity
            ├── Lambda network activity logs
            └── RDS login activity

Threat categories GuardDuty detects:
├── Reconnaissance:    port scanning, unusual API calls probing your
account
├── Instance compromise: crypto mining, C2 command-and-control callbacks
│   Example: "EC2 instance communicating with known malicious IP"
├── Account compromise:  credential exfiltration, root user unusual
activity
│   Example: "Root credentials used from unrecognized geographic location"
├── S3 threats:          unusual S3 enumeration, data exfiltration
patterns
│   Example: "S3 bucket contents listed from known Tor exit node IP"
├── Kubernetes threats:  anonymous API server access, privilege escalation
└── Malware:             EBS snapshot scanning (GuardDuty Malware
Protection)

Finding severity levels:
├── LOW:      1.0 – 3.9  (informational, low risk)
├── MEDIUM:   4.0 – 6.9  (investigate when time allows)
├── HIGH:     7.0 – 8.9  (investigate promptly)
└── CRITICAL: 9.0 – 10.0 (investigate immediately)

GuardDuty finding latency (minutes to hours) is intentional — ML models
need sufficient signal to avoid false positives. For real-time detection
of credential misuse, complement GuardDuty with: CloudTrail metric filters
on sensitive API calls, AWS Config rules on unusual access patterns, and
preventive controls (OIDC, SCPs) that reduce blast radius even when
detection is delayed.
```

```
# Enable GuardDuty (single-click, no log setup needed)
aws guardduty create-detector \
  --enable \
  --finding-publishing-frequency FIFTEEN_MINUTES

# Findings automatically flow to EventBridge
# EventBridge → SNS → PagerDuty / Lambda auto-remediation
```

```
GuardDuty Organizations integration:
├── Enable across all member accounts from delegated admin account
├── Aggregates findings from entire organization in one place
├── Member accounts cannot disable if admin has enforcement enabled
└── New accounts auto-enrolled when joining the organization

⚠ GuardDuty reads logs independently — you do NOT need to
   separately enable VPC Flow Logs or CloudTrail for GuardDuty.
   It reads these directly through its own internal access.
   Enabling GuardDuty is a single API call.
```

🧩 GuardDuty — Incident Response Playbook

```
When GuardDuty fires a High/Critical finding, treat it as a confirmed
breach until proven otherwise. The response follows five phases.

UnauthorizedAccess:IAMUser/MaliciousIPCaller — complete playbook:
An IAM user attempted an action from a suspicious IP address, and the
request was denied.
```

```
                    GuardDuty Incident Response
          UnauthorizedAccess : IAMUser/MaliciousIPCaller
 ──────────────────────────────────────────────────────────


   ┌──────────────────────────────┐
   │  Phase 0  ·  Triage  ·  5 min  │
   └──────────────────────────────┘
                    │
                    │
                    ├──▶  Extract from finding
                    │     IAM principal · source IP · API calls ·
timestamp
                    │
                    ▼
   ┌──────────────────────────────────┐
```

```
    │  Phase 1  ·  Containment  ·  15 min │
    └─────────────────────────────────────┘
                    │
                    ├──▶  Deny policy on identity
                    │     aws iam put-user-policy  /  attach deny all
                    │
                    ├──▶  Revoke active sessions
                    │     sts:RevokeSession  ·  rotate  /  delete keys
                    │
                    ▼
    ┌─────────────────────────────────────┐
    │  Phase 2  ·  Investigation  ·  1–2hr │
    └─────────────────────────────────────┘
                    │
                    ├──▶  CloudTrail scope query
                    │     All API calls by this principal  ·  all regions
                    │
                    ├──▶  S3 access logs  +  VPC Flow Logs
                    │     Data exfil check  ·  lateral movement
                    │
                    ▼
    ┌─────────────────────────────────────┐
    │  Phase 3  ·  Blast Radius Scoping   │
    └─────────────────────────────────────┘
                    │
                    ├──▶  Resources touched
                    │     EC2  ·  S3  ·  RDS  ·  Secrets Manager  ·  IAM
                    │
                    ├──▶  Lateral movement
                    │     New IAM users  ·  role assumptions  ·  backdoors
                    │
                    ▼
    ┌─────────────────────────────────────┐
    │  Phase 4  ·  Remediation & Harden   │
    └─────────────────────────────────────┘
                    │
                    ├──▶  Replace credentials  +  SCP guard
                    │     New keys  ·  MFA enforce  ·  deny bad IPs
                    │
                    └──▶  Post-incident review
                          Root cause  ·  detection gaps  ·  runbook update


  ──────────────────────────────────────────────────────────────────────
        ~0 – 4 hours  ·  finding to initial remediation
```

Phase 0 — Triage (First 5 Minutes)

Pull the full finding JSON — the console summary hides fields you need:

```
aws guardduty get-findings \
  --detector-id <detector-id> \
  --finding-ids <finding-id> \
  --query 'Findings[0]' \
  --output json
```

```
Critical fields to extract:
├── resource.accessKeyDetails.userName     → human, role, or service
account?
├── resource.accessKeyDetails.accessKeyId  → the exact key to revoke
├── service.action.awsApiCallAction.remoteIpDetails.ipAddressV4 → attacker
IP
├── service.action.awsApiCallAction.api    → which API call triggered the
finding
├── service.eventFirstSeen / eventLastSeen → sets CloudTrail query window
└── severity                               → 7–8 High, 9–10 Critical

First question: is this a human IAM user or a role/service account?
Containment path differs for each.
Also check: is this a scheduled red team or security scanner exercise?
If no scheduled exercise on the calendar → treat as real immediately.
```

Phase 1 — Containment (Within 15 Minutes)

```
Goal: stop the bleeding without destroying forensic evidence.
Do NOT delete anything during containment — only deny and revoke.
Deletion destroys CloudTrail correlation you will need in Phase 2.
```

```
# Step 1: Attach explicit Deny to the identity — instant, reversible
aws iam put-user-policy \
  --user-name <compromised-user> \
  --policy-name INCIDENT-RESPONSE-LOCKOUT \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [{"Effect": "Deny", "Action": "*", "Resource": "*"}]
  }'

# For a role: use permissions boundary instead
aws iam put-role-permissions-boundary \
  --role-name <compromised-role> \
  --permissions-boundary arn:aws:iam::aws:policy/AWSDenyAll

# Step 2: Revoke all active STS sessions (tokens may survive key
deactivation)
aws iam put-role-policy \
  --role-name <compromised-role> \
```

```
  --policy-name RevokeOldSessions \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "DateLessThan": {
          "aws:TokenIssueTime": "'$(date -u +%Y-%m-%dT%H:%M:%SZ)'"
        }
      }
    }]
  }'

# Step 3: Deactivate (not delete) the access key
aws iam update-access-key \
  --user-name <compromised-user> \
  --access-key-id <key-id> \
  --status Inactive

# Step 4: If EC2 instance role is involved — isolate to forensic SG
aws ec2 modify-instance-attribute \
  --instance-id <instance-id> \
  --groups <forensic-isolation-sg-id>
# forensic-isolation-sg: deny all inbound/outbound except your IR team's
IP
```

Phase 2 — Investigation (First 1–2 Hours)

```
Primary source: CloudTrail. 15-minute delivery lag — not real-time.
Query via Athena for speed at scale.
```

```
-- Everything this identity touched, all regions, full time window
SELECT eventtime, eventsource, eventname, awsregion,
       sourceipaddress, useragent, requestparameters, errorcode
FROM cloudtrail_logs
WHERE useridentity.arn LIKE '%<compromised-identity>%'
  AND eventtime >= '<first_seen_timestamp>'
ORDER BY eventtime ASC;

-- Privilege escalation: IAM write actions only
SELECT eventtime, eventname, requestparameters
FROM cloudtrail_logs
WHERE useridentity.arn LIKE '%<compromised-identity>%'
  AND eventsource = 'iam.amazonaws.com'
  AND eventname IN (
    'CreateUser','CreateAccessKey','AttachUserPolicy','PutUserPolicy',
    'CreateLoginProfile','UpdateLoginProfile','AddUserToGroup',
```

```
    'CreateRole','PassRole','AttachRolePolicy','PutRolePolicy'
  )
ORDER BY eventtime;

-- Data exfiltration signals: bulk S3 reads
SELECT eventtime, requestparameters, resources
FROM cloudtrail_logs
WHERE useridentity.arn LIKE '%<compromised-identity>%'
  AND eventsource = 's3.amazonaws.com'
  AND eventname IN ('GetObject','ListBucket','GetBucketPolicy')
ORDER BY eventtime;

-- Cross-account pivot: did they assume roles in other accounts?
SELECT eventtime, requestparameters, responseelements
FROM cloudtrail_logs
WHERE useridentity.arn LIKE '%<compromised-identity>%'
  AND eventname = 'AssumeRole'
ORDER BY eventtime;
```

Phase 3 — Blast Radius Scoping

```
IAM impact checklist:
  □ New IAM users created?              → list and delete
  □ New access keys created?            → deactivate all
  □ Policy changes made?                → diff and revert
  □ New roles with modified trust?      → audit trust relationships
  □ Console login profiles created?     → delete

Data impact checklist:
  □ S3 buckets listed or objects read?  → catalog accessed objects
  □ Secrets Manager secrets read?       → rotate ALL accessed secrets
  □ Parameter Store reads?              → rotate affected parameters
  □ KMS key usage?                      → determine what was decrypted
  □ RDS/DynamoDB queries?               → check DB audit logs

Compute impact checklist:
  □ EC2 instances launched?             → terminate, preserve AMI snapshot
  □ Lambda functions created/updated?   → diff code, check env vars for
exfil
  □ SSM Run Command executed?           → check command history

Network impact checklist:
  □ Security group rules modified?      → revert changes
  □ VPC peering or endpoints created?   → remove
  □ Route53 records changed?            → check for DNS hijacking

⚠ Priority: find PERSISTENCE MECHANISMS before remediating.
   Attackers often create backdoors 30+ minutes before the triggering
event.
   Missing a backdoor means the attacker re-enters after you lock them
out.
```

Phase 4 — Remediation and Hardening

```
# Generate fresh credentials after investigation is complete
aws iam create-access-key --user-name <user>

# Now safe to delete the compromised key (investigation is done)
aws iam delete-access-key \
  --user-name <user> \
  --access-key-id <old-key-id>

# Remove the emergency lockout policy
aws iam delete-user-policy \
  --user-name <user> \
  --policy-name INCIDENT-RESPONSE-LOCKOUT
```

```
// SCP: deny all API calls from the attacker's IP range
// Apply at OU level immediately — before closing the incident
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {
    "IpAddress": {
      "aws:SourceIp": ["<malicious-ip-cidr>"]
    }
  }
}
```

```
Hardening actions to complete before incident closure:
├── Enforce MFA on all IAM users if not already done
├── Migrate long-lived access keys to IAM Identity Center (SSO)
│   Long-lived keys are the root cause of most IAMUser findings
├── Add EventBridge rule to auto-trigger IR Lambda on future
│   GuardDuty High/Critical findings (automate Phase 1 containment)
└── Document: how did the key get exposed?
    git commit? CI secret? phishing? misconfigured application?
    Root cause determines the permanent fix.

When a public GitHub repo contains an AWS access key pattern, GitHub
automatically notifies AWS. AWS may proactively quarantine the key by
attaching an AWSCompromisedKeyQuarantine policy — which denies most
actions. If your key stops working and you find this policy attached,
check your public repos immediately.

Regulatory note:
If S3 buckets accessed contained PII or PHI:
```

```
→ GDPR 72-hour notification clock starts from confirmed breach date
→ HIPAA breach notification requirements may apply
→ Coordinate with legal/compliance before closing the ticket
```

```
⚠ Timing rule: complete Phase 3 (blast radius) BEFORE starting Phase 4.
   Remediating before you understand full scope means you will miss
   persistence mechanisms and declare incident closed prematurely.
```

🧩 AWS Security Hub

```
Security Hub = centralized security findings aggregator + compliance
checker

Two main functions:

1. Aggregate findings from multiple sources:
   ├── GuardDuty        (threat detection findings)
   ├── Amazon Inspector (vulnerability findings)
   ├── Amazon Macie     (sensitive data findings)
   ├── IAM Access Analyzer (external access findings)
   ├── Firewall Manager (WAF policy findings)
   ├── AWS Config       (compliance findings)
   └── Third-party tools: CrowdStrike, Palo Alto, Splunk, etc.

2. Run automated security standard checks:
   ├── CIS AWS Foundations Benchmark
   ├── AWS Foundational Security Best Practices (FSBP)
   ├── PCI DSS v3.2.1
   └── NIST SP 800-53 Rev. 5

All findings normalized to: AWS Security Finding Format (ASFF)
ASFF = standardized JSON schema for all security findings
```

```
# Enable Security Hub with default standards
aws securityhub enable-security-hub \
  --enable-default-standards   # enables CIS + FSBP automatically
```

```
Automated remediation pattern via Security Hub:
GuardDuty finding → Security Hub → EventBridge Rule → Lambda:

Finding: "EC2 instance communicating with malicious IP"
Lambda remediation:
  ├── Quarantine instance (modify SG to block all inbound/outbound)
```

```
    ├── Create EBS snapshot for forensic analysis
    └── Notify security team via SNS

Security Hub Organizations:
├── Designate a dedicated security account as delegated admin
├── All accounts auto-enroll and send findings
└── Central view of entire organization's security posture in one place
```

🧩 Amazon Inspector

```
Inspector = automated, continuous vulnerability assessment

Scans these resource types:
├── EC2 instances:       OS packages (via SSM agent — no separate agent)
├── ECR container images: on push + rescanned when new CVEs published
└── Lambda functions:    function code packages AND Lambda layers

Finding types:
├── Network reachability: are ports unintentionally exposed to internet?
└── Package vulnerabilities: known CVEs in OS or application packages

Inspector v2 (current generation):
├── Continuous rescanning as new CVEs are discovered daily
│   (not just on-demand scans — always monitoring your environment)
├── Prioritizes findings by: CVSS score × network reachability context
│   A critical CVE exposed to the internet = far more urgent than
│   the same CVE on an internal-only instance
└── Integrates with Security Hub for centralized finding management
```

```
# Enable Inspector for all resource types
aws inspector2 enable \
  --resource-types EC2 ECR LAMBDA LAMBDA_CODE
```

```
Inspector vs GuardDuty — critical distinction:
├── Inspector: PROACTIVE — finds vulnerabilities BEFORE exploitation
│              "This package has Log4Shell — patch it now"
└── GuardDuty: REACTIVE — detects active threats DURING exploitation
                "This instance is actively exploiting Log4Shell right now"

Use both: Inspector tells you what COULD be exploited.
          GuardDuty tells you what IS being exploited.
```

💬 Short Crisp Interview Answer

"GuardDuty, Security Hub, and Inspector are complementary security services forming a detection and response stack. GuardDuty is always-on ML-powered threat detection — it analyzes VPC Flow Logs, CloudTrail, and DNS logs to detect active threats like crypto mining, credential exfiltration, and C2 communication. Inspector is proactive vulnerability scanning — it continuously scans EC2, ECR images, and Lambda functions for CVEs and prioritizes findings by network reachability context. Security Hub is the aggregator — it collects findings from all these services plus third-party tools, normalizes them to the ASFF format, runs compliance checks against CIS and AWS best practice standards, and feeds findings to EventBridge for automated remediation. Inspector finds what COULD be exploited; GuardDuty finds what IS being exploited."

⚠ Tricky Edge Cases / Gotchas

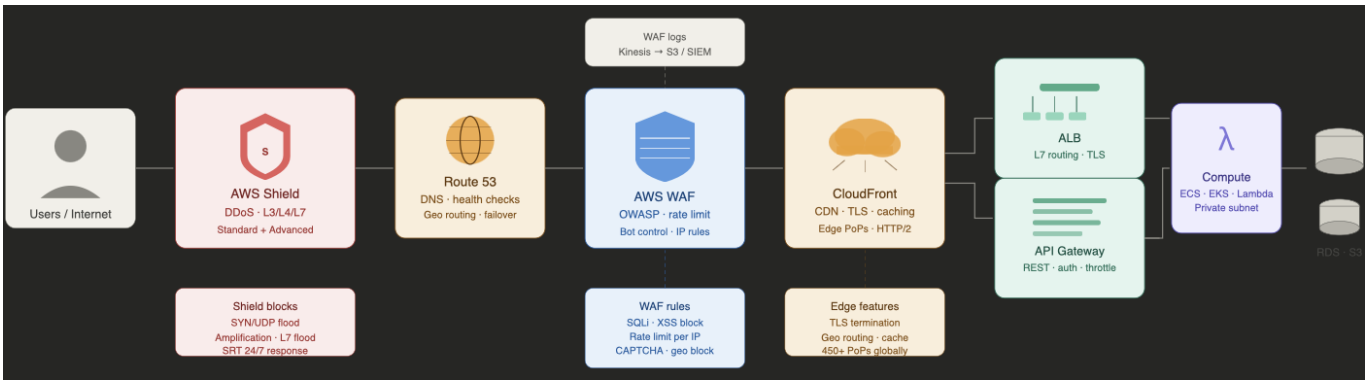
Gotcha	Detail
GuardDuty reads logs independently	No need to separately enable VPC Flow Logs or CloudTrail for GuardDuty
Inspector requires SSM agent	SSM agent must be installed and running on EC2 for package scanning
Security Hub = aggregator, not detector	Security Hub doesn't detect anything itself — it aggregates from other services
Finding suppression needs documentation	Can suppress findings in Security Hub, but document every suppression for audit purposes
Multi-account GuardDuty enforcement	Member accounts cannot disable GuardDuty when admin has enforcement enabled

- **Security Hub is an aggregator — it detects nothing itself** — it collects findings from GuardDuty, Inspector, Macie etc. Enabling Security Hub alone without enabling the source services produces no findings. Always enable source services organization-wide alongside Security Hub.
- **Cross-region aggregation must be explicitly configured** — Security Hub findings are regional by default. Without finding aggregation enabled, you need to check each region's Security Hub separately. Enable `ALL_REGIONS` aggregation for a true single pane of glass.
- **ASFF normalization is the key value proposition** — all findings from all sources use identical JSON schema. This enables unified EventBridge rules and Lambda remediation handlers that work regardless of which service generated the finding.

8.5 WAF & Shield

🟢 What It Is in Simple Terms

WAF (Web Application Firewall) filters malicious HTTP/HTTPS traffic at the application layer before it reaches your services. Shield protects against DDoS (Distributed Denial of Service) attacks at the network and transport layers. Together they form the complete protection layer for web-facing infrastructure.



🧩 AWS WAF

```
WAF protects these resources:
├── Application Load Balancer (ALB)
├── API Gateway (REST and HTTP APIs)
├── CloudFront distributions
├── AppSync GraphQL APIs
└── Amazon Cognito user pools

WAF components:
├── Web ACL:     the container for all rules (deployed to a resource)
├── Rules:       individual match + action logic
├── Rule Groups: reusable collections of rules
└── IP Sets:     named lists of IP addresses or CIDRs

Rule evaluation:
Rules evaluated in priority order (lower number = evaluated first)
First matching rule wins — evaluation stops at first match.
```

```
// Example WAF rule — block SQL injection in request body
{
  "Name": "BlockSQLInjection",
  "Priority": 1,
  "Statement": {
    "SqliMatchStatement": {
      "FieldToMatch": {"Body": {}},
      "TextTransformations": [{"Priority": 1, "Type": "URL_DECODE"}]
    }
  },
  "Action": {"Block": {}},
  "VisibilityConfig": {
    "SampledRequestsEnabled": true,
    "CloudWatchMetricsEnabled": true,
    "MetricName": "BlockSQLInjection"
  }
}
```

```
Match statement types:
├── IP-based:         IpSetReferenceStatement (allow/block IPs/CIDRs)
├── Geo-based:        GeoMatchStatement (block/allow by country)
├── String match:     ByteMatchStatement (match in headers, URI, body)
├── Regex:            RegexMatchStatement (pattern matching)
├── Rate-based:       RateBasedStatement (rate limit per IP over 5
minutes)
│   → count requests from a source IP in a 5-minute sliding window
│   → block if count exceeds your threshold
├── SQL injection:    SqliMatchStatement (OWASP SQLi protection)
├── XSS:              XssMatchStatement (cross-site scripting protection)
└── Size constraint:  SizeConstraintStatement (block oversized bodies)

AWS Managed Rule Groups (subscription-based, zero rule writing):
├── AWSManagedRulesCommonRuleSet:         OWASP Top 10 protections
├── AWSManagedRulesKnownBadInputsRuleSet: bad bots and malicious patterns
├── AWSManagedRulesSQLiRuleSet:           comprehensive SQL injection
├── AWSManagedRulesLinuxRuleSet:          Linux-specific exploit patterns
├── AWSManagedRulesAmazonIpReputationList: AWS-identified malicious IPs
└── AWSManagedRulesBotControlRuleSet:     bot detection + CAPTCHA
mitigation

WAF pricing:
├── $5/Web ACL/month
├── $1/Rule/month
└── $0.60/million requests inspected

⚠ WAF Web ACL for CloudFront MUST be created in us-east-1
   regardless of where your origin or users are located.
```

🧩 AWS Shield

```
Shield Standard (free, fully automatic):
├── Enabled on every AWS account automatically
├── Protects against: SYN/UDP floods, reflection/amplification attacks
├── Layer 3 and Layer 4 (network and transport) protection only
└── CloudFront and Route 53 receive additional DDoS protections
automatically

Shield Advanced ($3,000/month per organization + data transfer fees):
├── Enhanced DDoS protection for:
│   EC2, ELB, CloudFront, Route 53, Global Accelerator
├── DDoS Response Team (DRT): 24/7 AWS DDoS experts during active attack
├── Attack visibility: real-time metrics, attack vectors, historical
reports
├── Cost protection: credits for auto-scaling costs caused by DDoS attack
│   (if your ASG scaled up due to an attack — those costs are credited
back)
├── Proactive engagement: DRT contacts you during large-scale attacks
```

```
  └── WAF integration: DRT can write and apply WAF rules during active
  attacks

  Shield Standard vs Shield Advanced:
  ├── Standard: Layer 3/4 volumetric absorption (free, automatic)
  └── Advanced: Layer 7 protection via WAF + DRT support + cost protection

  Production defense architecture:
  CloudFront + WAF:      Layer 7 application filtering (SQLi, XSS, bots)
  Shield Advanced:       Layer 3/4 volumetric DDoS absorption
  WAF Rate-Based Rules:  secondary DDoS mitigation at application layer

  Rate-based rule as DDoS mitigation:
  WAF: block source IP if > 2,000 requests in 5 minutes
  → Legitimate user:  50 req/min → never triggered
  → DDoS bot:         10,000 req/min → blocked at WAF
  → Protects origin server while attack continues
```

💬 Short Crisp Interview Answer

"WAF operates at Layer 7 — it inspects HTTP/HTTPS request content and applies rules to allow, block, count, or CAPTCHA challenge requests. You build a Web ACL with rules for SQL injection, XSS, IP reputation, geo-blocking, and rate limiting. AWS Managed Rule Groups cover OWASP Top 10 with zero rule-writing effort. Shield Standard is automatic and free, protecting against Layer 3/4 volumetric attacks for all accounts. Shield Advanced at $3,000/month adds a 24/7 DDoS Response Team, detailed attack visibility, cost protection credits for auto-scaling triggered by DDoS, and proactive engagement. Production architecture is: CloudFront with WAF for application-layer filtering plus Shield Advanced for volumetric protection."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
WAF on supported resources only	Cannot attach directly to EC2 instances — must go through ALB, API GW, or CloudFront
Rule priority order	Lower number = evaluated first. Evaluation stops at first match — order is critical
Rate-based rules count all requests	Even COUNT action requests count toward rate-based thresholds
Shield Advanced per organization	One $3,000/month subscription covers the entire organization — not per account
WAF CloudFront must be us-east-1	Web ACL for CloudFront must be created in us-east-1 regardless of origin region

Gotcha	Detail
Shield Standard excludes DRT	DDoS Response Team access requires Shield Advanced — not included in Standard

---

8.6 AWS Config — Rules, Conformance Packs, Remediation

🟢 What It Is in Simple Terms

AWS Config is a continuous configuration recorder and compliance auditor. It tracks every configuration change to your AWS resources and evaluates them against rules. Instead of asking "did someone accidentally open this security group?" you can query the full change history and get alerted when any configuration drifts from your security standards.

🧩 AWS Config Core Concepts

```
AWS Config records three things:
├── Configuration snapshot:  complete state of all resources at a point in
time
├── Configuration history:   full log of all changes to a resource over
time
└── Configuration stream:    real-time change notification to SNS/S3

Config data flow:
Resource change → Config detects → Config Item created → S3 + SNS
notification

Config Item (what gets recorded for each resource):
{
  "configurationItemStatus": "OK",
  "resourceType": "AWS::EC2::SecurityGroup",
  "resourceId": "sg-0abc123",
  "configuration": {
    "groupName": "prod-web-sg",
    "ipPermissions": [
      {"ipProtocol": "-1", "ipRanges": [{"cidrIp": "0.0.0.0/0"}]}
      // ← Problem detected: all traffic from internet!
    ]
  },
  "configurationItemCaptureTime": "2024-01-15T14:30:00Z"
}

Config Rules evaluate compliance of recorded configurations:
├── AWS Managed Rules:    pre-built rules for common security standards
├── Custom Rules:         Lambda-backed rules for your own logic
```

```
└── Custom Policy Rules:  Config evaluation using Guard DSL (no Lambda
needed)

Rule evaluation triggers:
├── Configuration change: evaluate when the resource configuration changes
└── Periodic:             evaluate on a schedule (1hr/3hr/6hr/12hr/24hr)

Compliance status values:
├── COMPLIANT:       resource configuration meets the rule
├── NON_COMPLIANT:   resource configuration violates the rule
├── NOT_APPLICABLE:  rule does not apply to this resource type
└── ERROR:           rule evaluation itself failed
```

🧩 Config Rules — Common Examples

```
AWS Managed Rules (ready to enable immediately):
├── restricted-ssh:               SG must not allow unrestricted SSH
(0.0.0.0/0:22)
├── restricted-common-ports:      no unrestricted access to ports 22,
3389, 1433, 3306
├── s3-bucket-public-read-prohibited:   S3 buckets must not be publicly
readable
├── s3-bucket-versioning-enabled: S3 must have versioning enabled
├── s3-bucket-ssl-requests-only:  S3 must require HTTPS (deny HTTP)
├── ec2-instance-no-public-ip:    EC2 instances must not have public IPs
├── iam-root-access-key-check:    root user must not have active access
keys
├── iam-user-mfa-enabled:         all IAM users must have MFA configured
├── rds-instance-public-access-check: RDS instances must not be publicly
accessible
├── encrypted-volumes:            all EBS volumes must be encrypted
├── cloudtrail-enabled:           CloudTrail must be enabled in the
account
└── multi-region-cloudtrail-enabled: a multi-region trail must exist
```

```
# Create a custom Config rule (Lambda-backed)
aws config put-config-rule \
  --config-rule '{
    "ConfigRuleName": "require-s3-bucket-tag-environment",
    "Source": {
      "Owner": "CUSTOM_LAMBDA",
      "SourceIdentifier": "arn:aws:lambda:...:function:config-rule-s3-
tag",
      "SourceDetails": [{
        "EventSource": "aws.config",
        "MessageType": "ConfigurationItemChangeNotification"
      }]
    },
```

```
    "Scope": {
      "ComplianceResourceTypes": ["AWS::S3::Bucket"]
    }
  }'
# Lambda receives: configuration item + rule parameters
# Lambda must return: COMPLIANT / NON_COMPLIANT / NOT_APPLICABLE
```

🧩 Conformance Packs

```
Conformance Pack = collection of Config rules + remediation actions
                   packaged together for a complete compliance framework

Purpose: deploy 50+ rules at once for a compliance standard
         instead of creating each rule individually by hand

AWS sample conformance packs available:
├── Operational-Best-Practices-for-CIS-Level-1
├── Operational-Best-Practices-for-CIS-Level-2
├── Operational-Best-Practices-for-PCI-DSS
├── Operational-Best-Practices-for-HIPAA-Security
├── Operational-Best-Practices-for-NIST-CSF
└── AWS-Control-Tower-Detective-Guardrails
```

```
# Deploy a conformance pack
aws configservice put-conformance-pack \
  --conformance-pack-name "PCI-DSS-Compliance" \
  --template-s3-uri s3://my-bucket/pci-conformance-pack.yaml

# Deploy same pack across ALL accounts in org (from delegated admin)
aws configservice put-organization-conformance-pack \
  --organization-conformance-pack-name "Org-PCI-DSS" \
  --template-s3-uri s3://my-bucket/pci-conformance-pack.yaml \
  --delivery-s3-bucket my-config-bucket
```

🧩 Automated Remediation

```
Config Remediation = automatically fix non-compliant resource
configurations

Two remediation modes:
├── Manual:    trigger remediation from console on demand (safe for
critical resources)
└── Automatic: trigger immediately when NON_COMPLIANT status detected
```

```
Remediation executes via SSM Automation documents (runbooks).

Common remediation examples:
├── NON_COMPLIANT: S3 bucket public → Remediation: disable public access
block
├── NON_COMPLIANT: EBS unencrypted  → Remediation: create encrypted copy
├── NON_COMPLIANT: SG open SSH      → Remediation: remove 0.0.0.0/0:22
ingress rule
└── NON_COMPLIANT: no MFA on user   → Remediation: notify user + block
console
```

```
aws configservice put-remediation-configurations \
  --remediation-configurations '[{
    "ConfigRuleName": "restricted-ssh",
    "TargetType": "SSM_DOCUMENT",
    "TargetId": "AWS-DisablePublicAccessForSecurityGroup",
    "Parameters": {
      "AutomationAssumeRole": {
        "StaticValue": {"Values": ["arn:aws:iam::123:role/config-
remediation"]}
      },
      "GroupId": {
        "ResourceValue": {"Value": "RESOURCE_ID"}
      }
    },
    "Automatic": true,
    "MaximumAutomaticAttempts": 3,
    "RetryAttemptSeconds": 60
  }]'
```

```
⚠ Automatic remediation requires thorough testing before enabling.
   Wrong remediation logic = production outage (e.g., removing all SG
rules).
   Best practice: use manual remediation for critical resources.
   Consider: notify + document instead of auto-fix for high-risk
configurations.
```

💬 Short Crisp Interview Answer

"AWS Config continuously records the configuration state of every AWS resource and evaluates those configurations against rules. AWS Managed Rules cover common security checks like unrestricted SSH, public S3 buckets, and unencrypted EBS volumes — enable them in minutes. When a resource is non-compliant, Config can trigger automatic remediation using SSM Automation runbooks — for example, automatically removing an open SSH rule from a security group. Conformance packs bundle 50+ rules together for specific compliance frameworks like PCI-DSS or CIS Benchmarks and deploy across an entire organization with one command. The key value is drift detection and historical audit: you can query every

configuration change with a timestamp and prove compliance or identify the exact change that caused an incident."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Config records state, not API calls	CloudTrail records API calls. Config records resulting resource state. Both are needed
Config is not real-time	Configuration recording has a few minutes delay — not instant alerting
Auto-remediation can break production	Test thoroughly in non-production before enabling automatic mode
Config costs per configuration item	$0.003/item recorded. Large environments with frequent changes accumulate cost
Multi-account view needs aggregator	Config aggregator collects compliance data across accounts for org-wide reporting

- **Config records state, CloudTrail records events** — both are required for complete security posture — Config tells you WHAT the current and historical configuration is; CloudTrail tells you WHO changed it and WHEN. Neither alone is sufficient for a complete security investigation.
- **Config is not real-time** — configuration recording has a few minutes delay. It is not an alerting system for immediate threats — that is GuardDuty's role. Config is for compliance tracking and configuration history, not real-time threat detection.

8.7 IAM — ABAC, Session Policies

🟢 What It Is in Simple Terms

ABAC (Attribute-Based Access Control) lets you write policies using tags on principals and resources instead of listing specific resource ARNs. One policy rule automatically covers all resources with matching tags — no policy updates needed when you add new resources. Session policies let you further restrict permissions for a specific role assumption session — useful for temporary, scoped-down access.

🧩 ABAC — Attribute-Based Access Control

```
Traditional RBAC problem (Role-Based Access Control):
100 developers × 100 EC2 instances = 10,000 individual permission entries.
Every new resource requires a policy update.

ABAC solution:
```

```
Tag-based dynamic rules — one policy covers all combinations
automatically.

Tag EC2 instances:  {Team: payments, Environment: prod}
Tag IAM principals: {Team: payments, Environment: prod}
Policy condition:   allow access if resource tags match principal tags
```

```
// ABAC policy: allow EC2 management if tags match principal's tags
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["ec2:StartInstances", "ec2:StopInstances",
"ec2:DescribeInstances"],
    "Resource": "arn:aws:ec2:*:*:instance/*",
    "Condition": {
      "StringEquals": {
        "ec2:ResourceTag/Team":        "${aws:PrincipalTag/Team}",
        "ec2:ResourceTag/Environment": "${aws:PrincipalTag/Environment}"
      }
    }
  }]
}
```

```
Developer with IAM tags: Team=payments, Environment=prod
→ CAN manage: EC2 instances tagged {Team=payments, Environment=prod}
→ CANNOT touch: instances tagged {Team=platform, Environment=prod}
→ CANNOT touch: instances tagged {Team=payments, Environment=staging}

Adding a new team completely without policy changes:
1. Tag new IAM users/roles: Team=new-team
2. Tag new EC2 instances:   Team=new-team
3. Zero policy changes needed — policy already covers this case!
```

```
# Pass session tags when assuming a role (ABAC + STS)
aws sts assume-role \
  --role-arn arn:aws:iam::123:role/developer-role \
  --role-session-name alice \
  --tags Key=Team,Value=payments Key=Environment,Value=prod
# Session tags appear as:
# aws:PrincipalTag/Team = payments
# aws:PrincipalTag/Environment = prod
# Policy conditions referencing these tags work immediately
```

```
ABAC condition key support by service:
├── EC2:        ec2:ResourceTag/key        → instance-level support
├── S3:         s3:ResourceTag/key         → bucket-level only (not
object-level)
├── DynamoDB:   dynamodb:ResourceTag/key   → table-level support
└── RDS:        rds:db-tag/key             → instance-level support

⚠ ABAC requires BOTH resource tagging AND principal tagging.
   Untagged resources → condition never matches → no access granted.
   Enforce tagging with Config rules and SCP tag policies.
```

🧩 Session Policies

```
Session Policy = inline policy passed at sts:AssumeRole time
                 Further restricts the role's permissions for THIS session
only
                 CANNOT grant permissions beyond the role's existing
policies

Session policy intersection model:
Role identity policy:  Allow S3 + EC2 + DynamoDB (broad permissions)
Session policy:        Allow S3:GetObject on specific prefix only
Effective permissions: S3:GetObject on that prefix ONLY

┌──────────────────────────────────────────────────────────────┐
│  Role Identity Policy:  S3 + EC2 + DynamoDB                  │
│  Session Policy:        S3:GetObject on prod-bucket/reports │
│  Effective:             S3:GetObject on prod-bucket/reports │
│  (intersection — the most restrictive combination)           │
└──────────────────────────────────────────────────────────────┘

Use cases:
├── Temporary scoped-down task access:
│   "Alice needs developer-role access but only to prod-bucket today"
│   → Pass session policy restricting to that specific bucket + prefix
├── Vending machine pattern:
│   Mint just-in-time credentials scoped to exactly what's needed
└── Federated user scoping:
    Cognito user in "premium" group → session policy: access premium-
content/*
    Cognito user in "basic" group   → session policy: access basic-
content/* only
    One IAM role serves all users with different scoped sessions
```

```
# Assume role with session policy (restricts to specific S3 prefix)
aws sts assume-role \
  --role-arn arn:aws:iam::123:role/developer-role \
```

```
  --role-session-name alice-temp-access \
  --policy '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::prod-bucket/reports/*"
    }]
  }'
```

```
Session policy limits:
├── Cannot grant permissions beyond the role's identity policy
├── Maximum policy size: 2,048 characters (inline JSON)
└── Applies to current session only — next AssumeRole can differ

Full intersection model including all layers:
Effective = RoleIdentityPolicy ∩ SessionPolicy ∩ PermissionBoundary
All three layers must allow the action for it to succeed.
```

💬 Short Crisp Interview Answer

"ABAC shifts authorization from listing specific resource ARNs to tag-based dynamic matching. You tag IAM principals with Team=payments and resources with Team=payments, then write one policy condition: allow if ec2:ResourceTag/Team equals aws:PrincipalTag/Team. Adding new developers or new EC2 instances requires only tagging — zero policy changes. Session policies restrict permissions for a specific role assumption — they intersect with the role's identity policy so a broad developer role can be scoped to a single S3 prefix for a temporary task. The key constraint: session policies CANNOT grant more than the role already has — they only restrict further, never elevate."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
ABAC requires tag governance	Untagged resources get no access under ABAC. Enforce with Config + SCPs
Session policy cannot elevate	Can only restrict further — cannot grant what the role doesn't already have
S3 ABAC is bucket-level only	S3 ABAC conditions apply at bucket level, not individual object level
Session tags vs principal tags	Session tags set at AssumeRole time; principal tags set directly on IAM user/role
Transitive session tags	Set sts:TagSession permission to pass session tags through chained role assumptions

- `dynamodb:LeadingKeys` **is the DynamoDB-specific ABAC condition key** — it restricts access to items where the partition key matches the condition value. Combined with aws:PrincipalTag, this enables per-tenant data isolation with a single IAM role.
- **ABAC requires tag governance enforcement** — if your auth service fails to pass the correct session tag, the tenant either gets no access (if condition fails) or wrong access (if tag is missing and condition is not strict enough). Always validate tag presence before assuming the role.

8.8 KMS — Multi-Region Keys, Cross-Account Access, Grants

🧩 Multi-Region Keys

```
Problem: Encrypt data in us-east-1, need to decrypt in eu-west-1.
Standard CMK: key exists in ONE region only.
Cross-region decrypt requires: KMS call back to us-east-1 (latency +
complexity).

Multi-Region Key solution:
├── Primary key created in one region (e.g., us-east-1)
├── Replica keys created in additional regions (eu-west-1, ap-southeast-1)
├── All replicas share IDENTICAL key material (same cryptographic bytes)
├── Same key ID prefix across all regions (mrk-)
└── Data encrypted in us-east-1 decryptable in eu-west-1 locally!

Multi-Region Key ARNs:
Primary: arn:aws:kms:us-east-1:123:key/mrk-1234567890abcdef
Replica: arn:aws:kms:eu-west-1:123:key/mrk-1234567890abcdef
                                        ^^^^ identical mrk- key ID
```

```
# Create a replica key in eu-west-1
aws kms replicate-key \
  --key-id arn:aws:kms:us-east-1:123:key/mrk-1234 \
  --replica-region eu-west-1 \
  --key-policy file://eu-key-policy.json
```

```
Use cases for multi-region keys:
├── Global DynamoDB tables: same key encrypts/decrypts in all regions
├── Aurora Global Database: all regions use same key material
├── Disaster recovery: decrypt in DR region without cross-region API call
└── Active-active multi-region: each region encrypts and decrypts locally

⚠ Multi-region keys share key material across all regions.
```

```
    Compromise of key material compromises ALL regions simultaneously.
    Treat all replicas as a single key from a security risk perspective.
    Rotation: performed on primary — propagates to replicas automatically.
```

🧩 Cross-Account KMS Access

```
Method 1: Key Policy + IAM Policy (most common pattern)

Step 1: Key policy in key-owner account permits consumer account:
{
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::CONSUMER-ACCT:root"},
  "Action": ["kms:Decrypt", "kms:DescribeKey"],
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "kms:ViaService": "s3.us-east-1.amazonaws.com"
    }
  }
}

Step 2: IAM policy in consumer account grants the role permission:
{
  "Effect": "Allow",
  "Action": ["kms:Decrypt", "kms:DescribeKey"],
  "Resource": "arn:aws:kms:us-east-1:KEY-OWNER-ACCT:key/key-id"
}

Both steps are required. Neither alone is sufficient for cross-account
access.

Method 2: Grants (programmatic delegation without key policy changes)
```

```
# Create a grant — delegates specific operations to a grantee
aws kms create-grant \
  --key-id key-id \
  --grantee-principal arn:aws:iam::123:role/decryption-role \
  --operations Decrypt GenerateDataKey \
  --constraints '{"EncryptionContextEquals": {"service": "backup"}}'

# Retire a grant (grantee or retiring principal calls this)
aws kms retire-grant --key-id key-id --grant-token ...

# Revoke a grant (key admin calls this — immediate effect)
aws kms revoke-grant --key-id key-id --grant-id ...
```

```
Grant properties:
├── Grantee:           IAM principal who gets the delegated permission
├── Operations:        specific KMS actions allowed (Decrypt,
GenerateDataKey, etc.)
├── Constraints:       optional encryption context requirements
└── RetiringPrincipal: who is allowed to retire (self-expire) the grant

Grants vs key policy:
├── Key policy: requires key admin to modify (management overhead)
└── Grant:      created/retired programmatically at runtime (no admin
needed)

Use grants for: ephemeral access, service-to-service, per-resource
permissions
Use key policy for: persistent cross-account access, long-term service
access
```

🧩 Encryption Context

```
Encryption Context = additional authenticated data (AAD)
                     Key-value pairs passed during Encrypt
                     MUST be passed identically on Decrypt — or it fails

Purpose:
├── Binding: ciphertext can ONLY be decrypted with the correct context
├── Audit:   context appears in CloudTrail (what was decrypted and why)
└── Access:  key policies can require specific context values
```

```
# Encrypt with context
aws kms encrypt \
  --key-id key-id \
  --plaintext file://secret.txt \
  --encryption-context \
    service=backup,environment=prod,resourceArn=arn:aws:s3:::prod-bucket

# Decrypt MUST include exact same context key-value pairs
aws kms decrypt \
  --ciphertext-blob fileb://encrypted.bin \
  --encryption-context \
    service=backup,environment=prod,resourceArn=arn:aws:s3:::prod-bucket
# Wrong or missing context → Decrypt call FAILS with
InvalidCiphertextException
```

```
// Key policy requiring specific encryption context for Decrypt
{
```

```
    "Effect": "Allow",
    "Action": "kms:Decrypt",
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "kms:EncryptionContext:service":     "backup",
        "kms:EncryptionContext:environment": "prod"
      }
    }
  }
```

```
⚠ Encryption context is NOT a secret — it appears in CloudTrail logs.
   Use it for: audit trail, authorization binding, contextual integrity.
   Do NOT use it to store sensitive data (it is logged in plaintext).
```

💬 Short Crisp Interview Answer

"Multi-region KMS keys solve the cross-region decrypt problem — primary and replica keys share identical key material so data encrypted in us-east-1 can be decrypted locally in eu-west-1 without a round-trip API call. The trade-off is that key material compromise in one region affects all regions. For cross-account key access, you need both a key policy allowing the external account AND an IAM policy in the consumer account — both must allow the action. Grants offer a programmatic alternative: a key owner creates a grant at runtime delegating specific KMS operations to a grantee without modifying the key policy — ideal for ephemeral service-to-service access. Encryption context binds ciphertext to metadata — the exact same context must be provided on decrypt or the call fails, and it appears in CloudTrail for audit purposes."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Multi-region = shared material risk	Key material compromise in any region affects all replica regions
Cross-account needs both policies	Key policy alone is insufficient — IAM policy in consumer account also required
Grants are eventually consistent	After creating a grant, wait a few seconds before using it in your application
Encryption context must match exactly	Case-sensitive, exact key-value match. Missing any pair = decrypt failure
Grant retirement vs revocation	Retire = grantee or retiring principal calls it. Revoke = key admin calls it immediately

---

8.9 VPC Flow Logs + CloudTrail + Athena (Security Audit Pattern)

🟢 What It Is in Simple Terms

VPC Flow Logs capture network traffic metadata. CloudTrail captures every AWS API call. Together — queried with Athena using standard SQL — these form the foundation of AWS security auditing. After any incident you can reconstruct exactly what happened: what network traffic occurred and what API calls were made.

🧩 VPC Flow Logs

```
VPC Flow Logs capture network packet METADATA (not packet content):
├── Source IP address and port
├── Destination IP address and port
├── Protocol number (6=TCP, 17=UDP, 1=ICMP)
├── Bytes and packets transferred
├── Action: ACCEPT or REJECT
└── Start and end timestamps

Flow log record (default v2 format — space-delimited):
version account-id interface-id srcaddr dstaddr srcport dstport
protocol packets bytes start end action log-status

Example record:
2 123456789012 eni-0abc123 10.0.1.5 192.168.1.10 443 52341 6 10 4000
1705329000 1705329060 ACCEPT OK

Capture scope:
├── VPC level:    all traffic across the entire VPC
├── Subnet level: all traffic within a specific subnet
└── ENI level:    traffic on a single network interface

Destination options:
├── CloudWatch Logs: for Logs Insights queries and metric filter alerts
├── S3:              for Athena queries (cheaper long-term storage)
└── Kinesis Firehose: for real-time streaming and processing

Enhanced flow log fields (v5 format):
├── vpc-id, subnet-id, instance-id, az-id
├── pkt-srcaddr, pkt-dstaddr (original IPs for NAT-translated traffic)
├── flow-direction (ingress or egress)
└── traffic-path (which service the traffic traversed)

⚠ Flow logs capture network metadata ONLY — no packet payload content.
   You can see that a connection happened and how much data transferred.
   You cannot see WHAT data was transferred.
   REJECT entries: connection blocked by Security Group or NACL.
```

🧩 AWS CloudTrail

```
CloudTrail captures every AWS API call made in your account:
├── Who made the call    (IAM user, role, service — full ARN)
├── What call was made   (CreateBucket, PutObject, TerminateInstances)
├── When it happened     (exact timestamp)
├── From where           (source IP address, user agent / SDK used)
└── What was the result  (success or failure with error code)

Event types:
├── Management events (default, free):
│   Control plane operations: CreateEC2Instance, CreateS3Bucket,
PutBucketPolicy
└── Data events (additional cost $0.10/100K events):
    S3 object-level: GetObject, PutObject, DeleteObject
    Lambda:          Invoke
    DynamoDB:        GetItem, PutItem, DeleteItem

CloudTrail event example:
{
  "eventVersion": "1.08",
  "userIdentity": {
    "type": "AssumedRole",
    "principalId": "AIDABC123:alice",
    "arn": "arn:aws:sts::123:assumed-role/developer/alice",
    "accountId": "123456789012"
  },
  "eventTime":   "2024-01-15T14:30:00Z",
  "eventSource": "s3.amazonaws.com",
  "eventName":   "DeleteBucket",
  "awsRegion":   "us-east-1",
  "sourceIPAddress": "203.0.113.42",
  "requestParameters": {"bucketName": "prod-critical-data"},
  "errorCode":    null,
  "errorMessage": null
}

Trail types:
├── Single-region trail: captures events in one specific region
└── Multi-region trail:  captures all regions in one S3 bucket
    (organized by region prefix in S3 key path)

Organizations trail:
├── Created by management account — captures all member account events
└── Centralized, tamper-resistant audit log for entire organization

CloudTrail Insights:
├── ML model detects unusual spikes in API call rates
└── Example: "PutBucketPolicy called 500x in 5 minutes (baseline: 2x)"
```

🧩 Athena Security Audit Pattern

```
Architecture:
CloudTrail   → S3 (compressed JSON) → Athena → SQL queries → Security
findings
VPC Flow Logs → S3 (compressed text) → Athena → SQL queries → Network
analysis
```

```sql
-- Step 1: Create Athena table for CloudTrail logs
CREATE EXTERNAL TABLE cloudtrail_logs (
    eventVersion    STRING,
    userIdentity    STRUCT<
        type:STRING,
        principalId:STRING,
        arn:STRING,
        accountId:STRING
    >,
    eventTime       STRING,
    eventSource     STRING,
    eventName       STRING,
    awsRegion       STRING,
    sourceIPAddress STRING,
    requestParameters STRING,
    responseElements  STRING,
    errorCode         STRING,
    errorMessage      STRING
)
ROW FORMAT SERDE 'com.amazon.emr.hive.serde.CloudTrailSerde'
STORED AS INPUTFORMAT 'com.amazon.emr.cloudtrail.CloudTrailInputFormat'
OUTPUTFORMAT 'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION 's3://my-cloudtrail-bucket/AWSLogs/123456789012/CloudTrail/';

-- Security Query 1: Who deleted the S3 bucket?
SELECT eventTime, userIdentity.arn, sourceIPAddress, requestParameters
FROM cloudtrail_logs
WHERE eventName = 'DeleteBucket'
  AND requestParameters LIKE '%prod-critical-data%'
ORDER BY eventTime DESC;

-- Security Query 2: All root account activity (should be near zero)
SELECT eventTime, eventName, sourceIPAddress, userAgent
FROM cloudtrail_logs
WHERE userIdentity.type = 'Root'
ORDER BY eventTime DESC;

-- Security Query 3: Top failed API calls (attack or misconfiguration)
SELECT eventName, errorCode, COUNT(*) AS count,
       userIdentity.arn, sourceIPAddress
```

```sql
FROM cloudtrail_logs
WHERE errorCode IS NOT NULL
GROUP BY eventName, errorCode, userIdentity.arn, sourceIPAddress
ORDER BY count DESC;

-- Security Query 4: Console logins from external IPs
SELECT eventTime, userIdentity.arn, sourceIPAddress
FROM cloudtrail_logs
WHERE eventName = 'ConsoleLogin'
  AND eventSource = 'signin.amazonaws.com'
  AND NOT (sourceIPAddress LIKE '10.%' OR sourceIPAddress LIKE
'192.168.%')
ORDER BY eventTime DESC;
```

```sql
-- Step 2: Create Athena table for VPC Flow Logs
CREATE EXTERNAL TABLE vpc_flow_logs (
    version         INT,
    account         STRING,
    interfaceId     STRING,
    sourceAddress   STRING,
    destinationAddress STRING,
    sourcePort      INT,
    destinationPort INT,
    protocol        INT,
    numPackets      INT,
    numBytes        BIGINT,
    startTime       INT,
    endTime         INT,
    action          STRING,
    logStatus       STRING
)
STORED AS PARQUET
LOCATION 's3://my-flow-logs-bucket/AWSLogs/123456789012/vpcflowlogs/';

-- Network Query 1: Find port scanning / brute force attempts
SELECT sourceAddress, destinationPort,
       COUNT(*) AS rejected_attempts
FROM vpc_flow_logs
WHERE action = 'REJECT'
  AND destinationPort IN (22, 3389, 1433, 3306, 5432)
GROUP BY sourceAddress, destinationPort
ORDER BY rejected_attempts DESC
LIMIT 20;

-- Network Query 2: Largest data transfers (data exfiltration detection)
SELECT sourceAddress, destinationAddress,
       SUM(numBytes) AS total_bytes
FROM vpc_flow_logs
WHERE action = 'ACCEPT'
GROUP BY sourceAddress, destinationAddress
```

```
ORDER BY total_bytes DESC
LIMIT 20;
```

```
Athena performance and cost optimization:
├── Partition tables by year/month/day:
│   LOCATION 's3://bucket/logs/year=2024/month=01/day=15/'
│   Add: PARTITIONED BY (year STRING, month STRING, day STRING)
│   → Query last 24 hours = scan only 1 day of partitions (not all time!)
├── Convert to Parquet format:
│   4-10x faster queries + 4-10x cheaper than scanning raw JSON/text
└── Compress with Snappy or gzip:
    Further reduces storage and scan costs

Cost: $5/TB scanned
Unpartitioned, uncompressed: scanning 1 year of CloudTrail = $$$$
Partitioned Parquet:         scanning same data = 10-100x cheaper
```

💬 Short Crisp Interview Answer

"VPC Flow Logs capture network connection metadata — source/dest IP, port, protocol, bytes, and ACCEPT/REJECT — but no payload content. CloudTrail captures every AWS API call with full context: who, what, when, where, and result. Both ship to S3 and become queryable with Athena using standard SQL. The security audit pattern: after an incident, CloudTrail tells you exactly which IAM principal made suspicious API calls and from which IP. VPC Flow Logs show what network traffic accompanied those actions. Common queries: find all root activity, find external console logins, find top rejected traffic by port. Always partition your Athena tables by date and use Parquet format — unpartitioned scans cost $5/TB and a year of CloudTrail can be terabytes."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Flow logs = metadata only	No packet contents — you know connection facts but not what was transferred
CloudTrail data events cost extra	S3/Lambda/DynamoDB data events are $0.10/100K events — not free
CloudTrail ~15-minute delay	Events appear in S3 approximately 15 minutes after occurring — not real-time
No Athena partitions = expensive	Full table scans on unpartitioned CloudTrail = very high cost. Always partition
CloudTrail global events in us-east-1	IAM and Route 53 events are global — only captured in us-east-1 trail

Gotcha	Detail
Flow log REJECT source	REJECT from SG = stateful deny. REJECT from NACL = stateless deny. Different layers

---

8.10 Macie, Detective, Audit Manager

🟢 What It Is in Simple Terms

Three specialized security services for distinct problem areas. Macie discovers sensitive data (PII, financial data, credentials) inside S3 buckets. Detective investigates security incidents using graph analysis — mapping all related entities and activities. Audit Manager continuously collects compliance evidence so auditors have everything they need.

🧩 Amazon Macie

```
Macie = ML-powered sensitive data discovery service for S3

What sensitive data Macie finds:
├── PII:             names, SSNs, passport numbers, phone numbers, emails
├── Financial data:  credit card numbers, bank account numbers, routing
numbers
├── Health data:     medical record numbers, drug codes (HIPAA-relevant)
├── Credentials:     API keys, private keys, database connection strings
└── Custom patterns: your own regex patterns (for industry-specific IDs)

How Macie scans:
├── Samples and fully scans S3 object contents
├── ML models trained on millions of sensitive data examples
├── Reports: which bucket, which objects, which data types, how many
occurrences
└── Findings flow to Security Hub + EventBridge for automated response

Finding types:
├── Policy findings:          S3 configuration problems
│   Examples: public bucket, bucket shared externally, encryption disabled
└── Sensitive data findings:  actual sensitive data detected inside
objects
    Examples: 142 credit card numbers found in s3://prod-
logs/exports/jan.csv

Macie automated governance pattern:
Macie finding: "Sensitive data detected in prod-logs bucket"
→ EventBridge Rule → Lambda:
   ├── Tag object with Classification=Restricted
   ├── Move object to encrypted quarantine bucket
   └── Notify data governance team via SNS
```

```
# Enable Macie
aws macie2 enable-macie

# Create a scheduled classification job
aws macie2 create-classification-job \
  --name "prod-buckets-weekly-scan" \
  --job-type SCHEDULED \
  --schedule-frequency WEEKLY \
  --s3-job-definition '{
    "bucketDefinitions": [{
      "accountId": "123456789012",
      "buckets": ["prod-data", "prod-exports"]
    }]
  }'
```

```
⚠ Macie scans S3 object content — can be expensive for large buckets.
   Cost: approximately $1/GB scanned for data classification.
   Strategy: use sampling for initial discovery, targeted scans for
validation.
   Do not run continuous full scans on large buckets without cost
analysis.
```

🧩 Amazon Detective

```
Detective = graph-based security investigation tool
            Automatically collects and correlates security telemetry
            Builds interactive behavior graphs for incident investigation

Data sources Detective collects and correlates automatically:
├── VPC Flow Logs
├── CloudTrail management events
└── GuardDuty findings

Questions Detective answers instantly (vs hours of manual work):
├── Which IAM roles were used before and after this GuardDuty finding?
├── Which other resources did this potentially compromised role access?
├── What is the baseline behavior for this EC2 instance? What changed?
├── Which users logged in from this suspicious IP in the last 30 days?
└── What is the full blast radius of this credential compromise?

Detective vs manual investigation:
Without Detective:
→ GuardDuty: "EC2 instance communicating with known malicious IP"
→ Manual: query VPC Flow Logs + CloudTrail separately + correlate
timestamps
```

```
→ 2-4 hours of log archaeology to understand scope

With Detective:
→ GuardDuty finding → click "Investigate in Detective"
→ Graph: all connections to/from that IP in past 30 days
→ Related entities: IAM roles used, S3 buckets accessed, other EC2
instances
→ 5 minutes to understand the full incident scope
```

```
# Enable Detective (begins ingesting data automatically)
aws detective create-graph
# Detective immediately starts ingesting:
# VPC Flow Logs + CloudTrail + GuardDuty findings
# Graph analysis becomes useful after ~48 hours of data collection
```

```
Detective is NOT for:
├── Real-time alerting              (that is GuardDuty's job)
├── Vulnerability scanning          (that is Inspector's job)
└── Compliance checking             (that is Config + Security Hub's job)

Detective IS for:
└── Post-incident investigation:
    "Understand the full scope and timeline of what happened"

⚠ Detective requires approximately 48 hours of data
   before graph analysis becomes meaningfully useful.
```

🧩 AWS Audit Manager

```
Audit Manager = continuous automated compliance evidence collection

The problem it solves:
Annual SOC2 audit approaches.
Auditors need evidence: screenshots, logs, configs, policies.
Traditional approach: 2-3 months of manual evidence collection.

Audit Manager solution:
├── Define an audit framework (SOC2, PCI-DSS, HIPAA, GDPR, or custom)
├── For each control, specify data sources to collect evidence from:
│   - AWS Config rule evaluation results
│   - CloudTrail events matching control criteria
│   - Security Hub findings
│   - Custom evidence (manually uploaded documents)
├── Audit Manager continuously collects evidence automatically
└── Auditors receive: pre-organized evidence mapped to each control
```

```
Supported prebuilt frameworks:
├── SOC 2 Type II
├── PCI DSS v3.2.1
├── HIPAA Security Rule
├── GDPR
├── NIST Cybersecurity Framework
├── ISO 27001
└── Custom: define your own controls and evidence requirements

Types of evidence automatically collected:
├── Compliance checks:      Config rule evaluation results per resource
├── User activity:          CloudTrail events relevant to each control
├── Configuration snapshots: resource state at time of check
└── Manual evidence:         documents you upload (policy docs, process
docs)

Delegation workflow:
├── Assign specific controls to different team owners for review
├── Control owner reviews evidence and marks it as reviewed
├── Audit Manager owner generates final assessment report
└── Auditor receives complete, organized, pre-mapped evidence package
```

```
# Generate assessment report for external auditors
aws auditmanager get-assessment-report-url \
  --assessment-id abc-123-def \
  --region us-east-1
# Returns: pre-signed S3 URL valid for 2 hours to download complete report
```

💬 Short Crisp Interview Answer

"These three services address specific security specializations. Macie uses ML to discover sensitive data inside S3 objects — scanning content for PII, financial data, and credentials, then reporting which buckets and objects contain what type of sensitive data. It integrates with EventBridge for automated remediation like moving sensitive files to encrypted buckets. Detective is the security investigation tool — it builds interactive graphs correlating VPC Flow Logs, CloudTrail, and GuardDuty findings so when GuardDuty fires you can immediately see all related entities, the timeline, and full scope of a potential compromise in minutes instead of hours. Audit Manager automates compliance evidence collection — define SOC2 or PCI controls and it continuously pulls evidence from Config, CloudTrail, and Security Hub so when auditors arrive, everything is already organized and mapped to each control."

⚠ Tricky Edge Cases / Gotchas

Gotcha	Detail
Macie cost at scale	Large S3 buckets = significant cost at $1/GB scanned. Use sampling + scheduled scans

Gotcha	Detail
Detective 48-hour warmup	Graph analysis is not immediately useful — needs ~48 hours of data to build baseline
Detective ≠ real-time alerting	Post-incident investigation tool only. GuardDuty handles detection
Audit Manager evidence freshness	Evidence collected at specific points in time — ensure continuous collection for annual coverage
Macie custom identifiers	For industry-specific PII (employee IDs, medical record numbers), create custom regex identifiers

---

🔗 Category 8 — Full Connections Map

```
SECURITY connects to:

IAM
├── All AWS services     → every API call authenticated + authorized by
IAM
├── STS                  → AssumeRole, session tags, ExternalId
├── Organizations / SCPs → SCPs restrict IAM permissions org-wide (even
root)
├── KMS                  → key policies reference IAM principals
├── Config               → iam-root-access-key-check, iam-user-mfa-enabled
rules
└── CloudTrail           → all IAM changes logged as CloudTrail events
├── Access Analyzer      → monitors resource policies + CloudTrail to flag
external access and generate least-privilege policies

KMS
├── S3, EBS, RDS, DynamoDB, Lambda → all use KMS for encryption at rest
├── Secrets Manager      → secrets encrypted with KMS CMK
├── CloudWatch Logs      → log groups encrypted with KMS
├── Kinesis / SQS / SNS  → message-level encryption with KMS
└── CloudTrail           → trail log file encryption with KMS

Secrets Manager / Parameter Store
├── Lambda, ECS, EKS     → apps retrieve secrets at runtime (no
hardcoding)
├── RDS Proxy            → proxy retrieves DB credentials from Secrets
Manager
├── CodeBuild            → build-time credentials injection
└── KMS                  → secrets encrypted with KMS CMK

GuardDuty / Security Hub / Inspector
├── EventBridge          → findings trigger automated remediation
workflows
├── Lambda               → auto-remediation (isolate instance, block IP in
```

```
SG)
├── SNS                  → notify security team with finding details
├── S3                   → export findings for long-term analysis
└── Organizations        → centralized findings across all member accounts

WAF / Shield
├── CloudFront           → WAF protects CDN distributions (must be us-
east-1)
├── ALB                  → WAF attached to application load balancers
├── API Gateway          → WAF attached to REST and HTTP APIs
└── Route 53             → Shield protects DNS infrastructure

AWS Config
├── SNS                  → notify on non-compliance events
├── SSM Automation       → automated remediation runbooks
├── S3                   → configuration snapshots stored in S3
├── Security Hub         → Config compliance findings sent to Security Hub
└── Organizations        → org-level conformance packs across all accounts

CloudTrail
├── S3                   → trail logs stored in S3 (15-minute delay)
├── CloudWatch Logs      → real-time log monitoring + metric filter alerts
├── KMS                  → trail log file encryption at rest
├── SNS                  → real-time API event notifications
├── Athena               → SQL queries on CloudTrail log data in S3
└── EventBridge          → CloudTrail events trigger automation

Macie / Detective / Audit Manager
├── S3                   → Macie scans S3 object contents
├── GuardDuty            → Detective ingests GuardDuty findings for graphs
├── VPC Flow Logs        → Detective ingests for network graph analysis
├── CloudTrail           → Detective + Audit Manager both ingest
├── Security Hub         → Macie findings forwarded to Security Hub
└── Config               → Audit Manager pulls Config compliance evidence
```

📌 Quick Reference — Interview Cheat Sheet

Topic	Key Fact
IAM evaluation order	Explicit DENY → SCPs → Resource policy → Permission boundary → Session policy → Identity policy
Explicit DENY	Always wins — cannot be overridden by any Allow in any policy
SCPs restrict root	Even account root user is blocked by SCPs from parent OU
Permission boundary	Caps what identity policy can grant. Effective = identity ∩ boundary
Trust policy	Resource-based policy ON the role. Defines who can call sts:AssumeRole
ExternalId purpose	Confused deputy protection for cross-account third-party access

Topic	Key Fact
SCPs don't apply to	Management account, service-linked roles, AWS service operations
KMS key policy	Mandatory for CMKs. No key policy = no access, including for account admins
AWS managed keys	Free, auto-rotated annually, key policy not modifiable by you
Customer managed keys	$1/key/month, custom key policy, optional rotation
Envelope encryption	GenerateDataKey → encrypt locally → store encrypted DEK alongside ciphertext
KMS direct encrypt limit	4KB maximum. Use envelope encryption for larger data
Key deletion minimum wait	7 days — cannot be shortened. Deletion is irreversible
Secrets Manager rotation	4 Lambda steps: createSecret → setSecret → testSecret → finishSecret
Secret version stages	AWSPENDING (new), AWSCURRENT (live), AWSPREVIOUS (prior)
Parameter Store tiers	Standard = free/4KB. Advanced = $0.05/param/8KB + parameter policies
GuardDuty data sources	VPC Flow Logs, CloudTrail, DNS, EKS audit, RDS login, Lambda network
GuardDuty vs Inspector	GuardDuty = active threat detection. Inspector = proactive vulnerability scanning
Security Hub	Aggregates findings from all services, normalizes to ASFF, runs CIS/FSBP checks
Inspector EC2 requirement	SSM agent must be running on EC2 for package vulnerability scanning
WAF supported resources	ALB, API Gateway, CloudFront, AppSync, Cognito user pools
WAF rule evaluation	Priority order — lower number evaluated first. First match wins
WAF CloudFront requirement	Web ACL for CloudFront must be created in us-east-1
Shield Standard	Free, automatic, Layer 3/4 only for all AWS accounts
Shield Advanced	$3,000/month organization. DRT access, cost protection, proactive engagement

Topic	Key Fact
Config vs CloudTrail	Config = resource configuration state history. CloudTrail = API call audit log
Config auto-remediation	Uses SSM Automation runbooks. Test thoroughly before enabling automatic mode
Conformance packs	Bundle 50+ Config rules for compliance frameworks (PCI, CIS, HIPAA)
ABAC requirement	Both resource AND principal must be tagged. Untagged resources = no match = no access
Session policy rule	Cannot elevate beyond role identity policy — only restricts further
Multi-region KMS keys	Same key material across all regions. Encrypt in us-east-1, decrypt locally in eu-west-1
KMS grants	Programmatic delegation without modifying key policy. Use for ephemeral access
Encryption context	Must match exactly on decrypt (case-sensitive). Appears in CloudTrail — not a secret
VPC Flow Logs content	Network metadata only (IPs, ports, bytes, ACCEPT/REJECT). No packet payload
CloudTrail delay	~15 minutes for events to appear in S3 bucket
Athena cost savings	Partition by date + Parquet format → 10-100x reduction in scan cost
Macie purpose	ML-powered PII and sensitive data discovery inside S3 object contents
Macie cost	~$1/GB of S3 data scanned. Use sampling for large buckets
Detective purpose	Graph-based post-incident investigation. Maps all related entities and timeline
Detective warmup	Needs ~48 hours of data before graph analysis is meaningfully useful
Audit Manager	Continuous compliance evidence collection for SOC2/PCI/HIPAA/GDPR audits
IAM Access Analyzer	Finds resources accessible from OUTSIDE zone of trust (account or org). Monitors S3, IAM roles, KMS, Lambda, SQS, Secrets Manager, SNS
Access Analyzer zone of trust	Account = external is outside this account. Organization = external is outside the org
Access Analyzer policy generation	Analyzes 90 days of CloudTrail to generate least-privilege policy from actual usage
IAM Access Advisor	Shows last-accessed timestamps per service/action for a principal. Use for right-sizing and removing unused permissions
Access Analyzer vs Access Advisor	Analyzer = inbound external exposure. Advisor = outbound usage analysis. Both serve least-privilege goals

Category 8: Security & Compliance — Complete Interview Guide
Excluded topics: CodeBuild, CodePipeline, CloudFormation